JSS MAHAVIDYAPEETHA

JSS Science and Technology University

"Automated CMOS Logic and Layout Visualization Web Application-Based Tool"

A technical project report submitted in partial fulfillment of the award of the degree of

BACHELOR OF ENGINEERING

IN

ELECTRONICS AND COMMUNICATION ENGINEERING

BY

A Siddhartha	**01JCE21EC001**
Jayanth K S	**01JST21EC017**
Nikhil Naik	**01JCE21EC027**
Sathvik M	**01JST21EC035**

Under the guidance of
Dr. M G Veena
Professor
Department of Electronics and Communication Engineering
SJCE, JSS S&TU, Mysuru.

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING
2025-2026

JSS MAHAVIDYAPEETHA

JSS Science and Technology University

CERTIFICATE

This is to certify that the work entitled "Automated CMOS Logic and Layout Visualization Web Application-Based Tool" is a Bonafide work carried out by A Siddhartha, Jayanth K S, Sathvik M, Nikhil Naik in partial fulfilment of the award of the degree of Bachelor of Engineering in Electronics and Communication Engineering for the award of Bachelor of Engineering by JSS Science and Technology University, Mysuru, during the year 2025-2026. The project report has been approved as it satisfies the academic requirements in respect to project work prescribed for the Bachelor of Engineering degree in Electronics and Communication Engineering.

Under the guidance of

Dr. M G Veena
Professor,
Department of ECE, SJCE,
JSS STU, Mysuru.

Head of the Department

Dr. Sudarshan Patil Kulkarni,
Professor and Head,
Department of ECE, SJCE,
JSS STU, Mysuru.

Examiners:

1. .....
2. .....
3. .....

FYP report

ORIGINALITY REPORT

9%	8%	7%	6%
SIMILARITY INDEX	INTERNET SOURCES	PUBLICATIONS	STUDENT PAPERS

PRIMARY SOURCES

1	Submitted to King's College Student Paper	1%
2	hal.archives-ouvertes.fr Internet Source	1%
3	etd.lib.metu.edu.tr Internet Source	<1%
4	Submitted to University of Liverpool Student Paper	<1%
5	Submitted to University of Derby Student Paper	<1%
6	A. Ahmad, D. Ruelens, S. Ahmad, L. Pathuri. "Determining the possible minimal Boolean expressions via a newly developed procedure and tool", 2015 6th International Conference on Computing, Communication and Networking Technologies (ICCCNT), 2015 Publication	<1%
7	arxiv.org Internet Source	<1%
8	core.ac.uk Internet Source	<1%
9	Vojin G. Oklobdzija. "Digital Design and Fabrication", CRC Press, 2017	<1%

DECLARATION

We do hereby declare that the project titled "Automated CMOS Logic and Layout Visualization Web Application-Based Tool" is carried out by the project group, under the guidance of **Dr. M G Veena**, Professor, Department of Electronics and Communication Engineering, JSS Science and Technology University, Mysuru, in partial fulfilment of requirement for the award of Bachelor of Engineering by JSS Science and Technology University, Mysuru, during the year 2025-2026.

We also declare that we have not submitted this dissertation to any other university for the award of any degree or diploma course.

Date:

Place: Mysuru

A Siddhartha

Jayanth K S

Nikhil Naik

Sathvik M

ACKNOWLEDGEMENT

The success and the final outcome of this project required a lot of guidance and assistance and we are privileged to have got all these along with the completion of this project.

We are grateful to our Project guide **Dr. M G Veena**, Professor, Department of Electronics and Communication Engineering, JSS Science and Technology University, SJCE, Mysuru for her constant support and encouragement towards the completion of this project.

ABSTRACT

CMOS VSLI Logic Visualizer is an online education and design tool designed for easier understanding, generation and visualization of circuit type CMOS transistor level circuits from a given Boolean expression. Very Large-Scale Integration (VLSI) is playing an ever more important role in recently used Digital electronics and it is important to provide tools to convey both the connection between Boolean logic and CMOS implementation, and the connection between the generated CMOS netlist and actual CMOS layout generation. Interpretation of the transistor arrangement and layout can be difficult, especially for an inexperienced designer, while traditional CMOS design methods involve manual tasks. To solve these problems, the proposed system consists of an interactive, web-based application combining Boolean logic processing, CMOS circuit generation, layout visualization and simulation.

Tools and technologies being used in the project are React for front-end, TypeScript for logic, CSS/HTML/SVG for graphics, Vite to build it and develop it, and Node for managing things. Modular structure of the application allows to easily integrate the front and the back-end via the function calls that are supported with TypeScript and shared data objects. The system offers quick processing, re-usability of components, metal polygonal architecture and dynamic real-time rendering. The CMOS VLSI Logic Visualizer is proposed as a new economical, scalable and easy to use platform for training and study of the concepts related to CMOS circuit designing. The application provides an extensive benefit of making CMOS transistor level implementation and VLSI layout generation very simple process. It is well suited to use in engineering education, in the VLSI laboratory training, in the learning of digital electronics and in the analysis of the basic CMOS circuit design.

Keywords: CMOS Logic Visualizer, Pull-Up Network (PUN), Pull-Down Network (PDN), Stick Diagram, Layout Generator, Logic Processing Engine, SVG Visualization, Frontend-Backend Integration.

Contents

List of Figures

List of Tables

Chapter 1
Introduction

In the rapidly evolving field of electronics and semiconductor technology, CMOS (Complementary Metal-Oxide Semiconductor) logic has emerged as the dominant technology for implementing digital integrated circuits. The electronics sector has experienced immense growth through the advancements in semiconductor technologies. Among the key technologies, the use of CMOS logic is predominant in the design of digital ICs in microprocessors, memories, and embedded systems. With the rising complexity in digital systems, knowledge of CMOS logic becomes inevitable for any students, researcher, or engineer. The learning process is difficult as there are multiple layers of abstraction involved from Boolean expressions, logical gates, to transistors design, and layout implementation. The current method of teaching CMOS uses static images and explanations which cannot adequately explain the process of designing CMOS logic circuit from logical expression to its implementation on the circuit layout.

This research project is aimed at creating a web application for visualization of CMOS circuits and layouts for Boolean expressions. In the course of development, the main goal would be to create a platform capable of taking logical expressions as inputs and producing their circuit designs with pull up and pull-down networks and the layout design as well. This tool will enable an easy visual observation of how digital logic is implemented physically at the transistor level and even in simple circuit layout. The knowledge of CMOS logic becomes inevitable for any students, researcher, or engineer. The learning process is difficult as there are multiple layers of abstraction involved from Boolean expressions, logical gates, to transistors design, and layout implementation. By using the software we get error less output, it is easy to understand and implement by any aged person.

1.1 Preamble

The preamble acts as an introductory foundation towards the general discipline within which the project is based. In relation to the field of modern electronics, the rapid growth in the development of semiconductor technology has had a great impact on digital systems. The development in the realm of digital electronics can be largely attributed to the growth in the use of CMOS (complementary metal-oxide semiconductor). CMOS technology offers low power dissipation, noise immunity and scalability hence it forms the basis of design for all digital integrated circuits. CMOS is used in microprocessors, memories, embedded systems and consumer electronics therefore making it an important discipline in the fields of electronics and computer engineering [1].

The growth in the complexity of digital systems has greatly impacted on the design of digital circuits. The task of designing circuits requires designers to go past logical design of gates and to understand how circuits function at the level of transistors. CMOS logic uses a complementary set of PMOS (positive channel metal-oxide-semiconductor) and NMOS (negative channel metal-oxide-semiconductor) transistors. This allows for efficient switching characteristics and low power dissipation in the circuit. The pull-up network uses PMOS transistors while the pull-down network uses NMOS transistors. These two networks help to achieve the desired Boolean functions. Comprehension of the relationship between the two networks becomes important when analysing the performance of the circuit [2].

Alongside circuit design, the layout design of CMOS circuits is equally important. In CMOS design, the task of layout design entails the positioning of transistors and connections along with various layers including diffused layers, polysilicon layers and metal layers. The process of layout design takes into consideration different design rules. As a result of going from logical design to physical design, some issues arise hence need for thorough comprehension of the logical and physical design in CMOS [3-4].

1.2 Motivation

The inspiration for this project comes from the difficulties encountered by beginners and students in comprehending the principles of CMOS logic design. First, there is a problem associated with the visualisation of Boolean logic and conversion of expressions into transistor circuits. Although books give a detailed description and diagrams of such conversion processes, there is a lack of an interactive environment where one could test different input values and see how this affects the circuit design. Secondly, there is a lack of easy-to-use software which can be employed for both logic design and transistor layout viewing. The software that is currently used in semiconductor companies for these purposes is complex, expensive and requires special training.

Therefore, a beginner does not have access to such programs. In addition, the importance of VLSI design in the current technological world increases every day, which calls for the necessity to have intuitive educational materials and programs for this sphere. Thus, this project can be considered useful because of its potential for improving education and promoting the development of this field. Moreover, simulating logic behaviour and visualising transistor switching will improve understanding of digital circuits.

1.3 Problem Statement

CMOS logic circuit design based on Boolean expressions is a difficult and tedious task that involves a great deal of knowledge about logic circuit design as well as transistor circuit design. However, there are no current tools that can easily convert a Boolean expression to a CMOS circuit, show the connections at the transistor level, generate layouts, and simulate the circuit behaviour. Hence, this poses a problem not only in educational purposes but also during the initial stages of designing the circuit itself. In order to solve this problem, the aim of this project is to create a web-based CMOS logic circuit visualizer and layout generator.

1.4 Block Diagram

This block diagram of the CMOS Logic Circuit Visualizer and Layout Web Application contains all the blocks that saw in the process of transforming a Boolean expression into a transistor-level circuit, a layout structure, and the output signal simulated for the circuit. To begin, the user types into the Boolean Expression Input Module a logic expression, the process starts with the Boolean Expression Input Module, in which the user enters a logic expression.

This module is used to connect the user and the application, while guaranteeing that the expression entered is in a valid syntax. The expression then moves to the Expression Parser Module that evaluates the Boolean equation and returns a syntax tree that is structured. In this phase, logical operations (AND, OR, NOT) are detected and organized by the order that they are executed. The parser also alerts to errors in the syntax like any type of invalid symbols or missing brackets, which makes sure that it is processed correctly.

Once the expression has been parsed, it is passed on to the Logic Processing Engine in which the Boolean algebra laws are used in conjunction with optimization techniques to simplify the expression. This step simplifies the circuit by performing only required operations and minimizing the number of required transistors to implement the circuit. The simplified expression is then fed into the CMOS Circuit Generator Module which automatically generates both the Pull-Down Network (NMOS) and Pull-Up Network (PMOS).

The system uses a Boolean logic to determine whether the transistors need to be in series or parallel. The LOW output condition is controlled by the NMOS network, and the PMOS network controls the HIGH output condition. This stage is useful to make the user familiar with the conversion of the logic expressions to actual transistors level CMOS circuit.

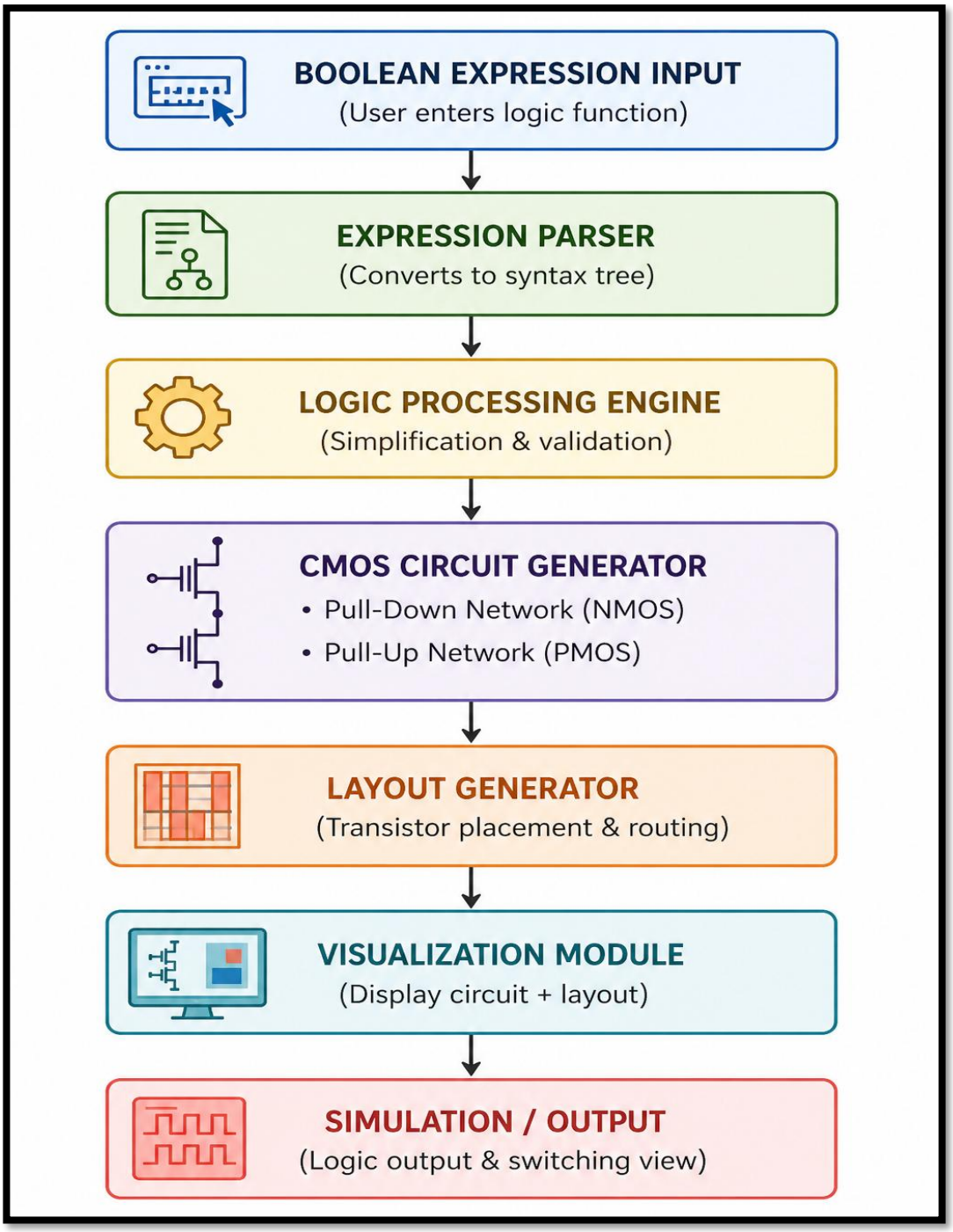


Figure 1.4.1: The Proposed System Architecture.

1.5 Objectives

The primary objectives are:

1. To design a system which transforms Boolean expressions to CMOS transistor level circuit in an automatic manner with the support of generating pull up network (PMOS) and pull-down network (NMOS) simplifying this design process to the user.

2. Build an interactive visualization system for helping the users to understand the connection of the transistors, circuit structure and to clearly see the CMOS circuit formed and the corresponding architecture.

3. Implement a real-time simulation system which consumes the logic output and enables users to analyze the circuit behavior and verify whether the logic designed for CMOS is correct.

1.6 Organization of the report

The report is structured to make the understanding of the proposed CMOS Logic Circuit Visualizer and Layout Web Application easy.

Chapter 1's introduction and background of CMOS technology, the importance of the issue of visualization tools, the motivation for the work, the problem statement, and goals of the system.

Chapter 2 has a detailed literature survey of presently available tools and research in the areas of digital logic design, CMOS circuit generation, and visualization systems. It also includes an overview of the weaknesses of existing methods and a statement of a research requirement which this project will resolve.

Chapter 3 proposes solution system design and architecture is discussed here. It describes all the workflow, the block diagram, as well as the different modules like the expression parser, logic processing engineer, the CMOS circuit generator and the layout generator.

Chapter 4 investigates the implementation specificities of the system like technologies and algorithms used for the parsing of a Boolean formula and generation of a circuit, the design of the front-end and back-end parts of the system.

Chapter 5 presents the results and discussion of the study and the conclusion and future scope of the project are presented in Chapter 5. It checks the performance of the system, explains the pros and cons of the system and recommends possible improvements for future development.

Chapter 2

Literature Review with Gap in the Literature

A topic of recent research interest in digital electronics and VLSI design is developing software tools and automation techniques to enhance the understanding, design and visualization of CMOS logic circuits. There are many existing works existing on individual issues like logic simulation, circuit design or layout generation. But very few systems have these components working together in a single system. Some of the tools specialize in Boolean logic simplification, some are independent and can simulate transistor level, and some are for layout design. Consequently, there is a lack of work in the area of an integrated system linking the following: Boolean expression, CMOS circuit generation, layout visualization, and real-time simulation. Hence, in this chapter, the relevant works in the field are reviewed and its requirements for complete CMOS logic visualization platform are highlighted.

2.1 Previous research

Early developments in digital electronics education introduced basic logic simulators that allowed users to design circuits using standard logic gates such as AND, OR, and NOT. These tools provided a fundamental understanding of digital logic behaviour; however, they operated at an abstract level and did not include transistor-level representation. As a result, users were unable to visualize how logic gates are physically implemented in CMOS technology [1].

Subsequent research led to the development of SPICE-based simulators, which provided detailed transistor-level analysis of circuits. These simulators enabled accurate modelling of MOSFET behaviour, including switching characteristics. Although highly accurate, SPICE tools require users to write complex netlists and interpret simulation waveforms, making them unsuitable for beginners and educational environments [2].

It is suggested that interactive learning tool should be used for better understanding of digital circuits using graphical interfaces. These tools offer visual representations of the operation of the circuits, allowing the user to see the signal flow and logic behaviour as they interact with the circuit. However, these systems work at the gate level only and are not expanded into the network of CMOS transistors [3].

There has been a considerable research effort into parsing Boolean expressions, which has helped generate automated circuits. A logical expression could be transformed to a structured form, such as a syntax tree, with the aid of parsing techniques, which would facilitate its efficient processing and transformation. These are the basic techniques used in systems designed to attempt to automate the process of circuit design from high level specification [4].

The methods of Boolean simplification like Karnaugh map and Quine-McCluskey algorithm have been widely investigated to minimize logic expressions. Such techniques are used at a logic level and are not directly implementable on a transistor level [5].

The theory of CMOS designing principle has been insufficient for efficient switching. The use of pull-up networks (PMOS) and pull-down networks (NMOS) accounts for the fact that the consumption of power by CMOS circuits is very small and reliable operation is also guaranteed. These principles are the principles of modern digital circuit design [6].

Modelling of the MOSFET devices has been studied, emphasizing the operation of the devices under various conditions. These studies are very important for the study of the performance parameter in CMOS circuits like propagation delay, leakage current and dissipation power [7].

There have been some publications devoted to method of layout design, which stress the effects of the physical layout on circuit performance. The layout design includes the precise placement of the transistors and interconnections requiring the minimum area and delay. The techniques, however, are more advanced and sophisticated ones and commonly require dedicated equipment and skills [8].

Graphical layout editors have been developed that help designers to produce integrated circuit designs. These are very powerful development tools with design features but are normally only utilized within industry where a fundamental understanding of the fabrication rules is necessary and where the beginners are not as successful [9].

The last few years have seen dramatic progress in web technology that has resulted in the creation of browser-based sim tools. These also offer accessibility and ease of use, with users being able to interact with circuits without having to install any specific software. Most of the web-based tools are however available to simulate logic and not implement CMOS [10].

Backend frameworks such as Flask and Django have been widely used for developing web-based applications. These frameworks can help with implementing logic processing, data management and communication between front and back-end elements to create an interactive system [11].

Methods of visualization thinking have been proposed as a means towards better representation of complex systems. The graphical representation of circuits has been found to be useful in comprehending their structure and behaviour particularly in conjunction with interactive aspects [12].

Recently machine learning applications have been used in circuit design to optimize and diagnose faults. These techniques are mainly focused on performance improvement and

are not directly related to educational visualization tools [13].

VLSI educational programs have been created that can be used in teaching concepts like logic synthesis and layout design. However, a great deal of this offers static information and also do not have interactivity which makes them not as efficient in conveying the practical concepts [14].

Research in logic synthesis has enabled the development of processes for transforming high level descriptions into hardware implementations. These techniques have been

commonly applied in digital design automation, but they're complicated and may be not very convenient to use by novice user [15].

Extensive studies on simulation techniques have been carried out, to assess circuit behaviour under different conditions. Real-time simulation is especially significant to grasp the working of the circuit at runtime and to check its correctness [16].

Research in user interface design focuses on the usability of interfaces concerning enhancement of learning outcome. To enhance the engagement and understanding of the users, it is essential that the interfaces are well designed [17].

The use of cloud-based platforms for hosting the simulation tools, with scalability and remote access capabilities has been investigated. These platforms allow to access any tool wherever you go, but can be missing of some specific CMOS visualization tools [18].

Interactive approach that uses software to help understand complex topics like digital electronics and VLSI design has proved to be effective in a great deal of research. These tools allow to learn and experiment manually [19].

Several attempts have been made to combine the various functions on a single platform, but it is difficult to integrate all aspects of the logic design, circuit generation, and visualization of the layout to achieve optimal integration [20].

Burton et al. Have, however, found that these systems tend to be lacking in visualization and user-friendliness [21].

To aid in expressing program logic graphically, graphical programming environments have been created. There is evidence that visualization is a useful tool in the understanding of complex systems in contexts like these [22].

The development of dynamic and interactive visualizations is made possible by the use of modern JavaScript libraries like D3.js and Canvas. They are useful for particular applications where circuit diagrams are needed, such as in web applications [23].

Ensuring scalable and maintainable systems is critical to the use of modular architecture as demonstrated in research on design and analysis of modular architecture. Design and analysis of modular architecture reveal the importance of the use of this concept in creating scalable and maintainable systems. The modular designs enable efficient development and integration of various module components [24].

The focus on using integrated educational tools that fuse logic design, transistor level visualizations, layout generation, and simulation, has been pulled into focus through recent research. The tools could be applied to connect theoretical knowledge to applications [25].

2.2 Summary of Literature Review

From the literature search the following conclusions can be drawn: Development in the areas of Digital Logic Simulation, CMOS circuit design, circuit layout and visualization technology, is a well-established development and in almost all systems developed so far only one aspect is treated. Thus far, there is no overall solution. Logic simulators allow gaining insight at a high level but cannot simulate transistors, whereas circuit simulators are quite accurate but difficult to operate. While it can be helpful, layout applications are only useful if one has certain professional skills but they are not capable of creating circuits or circuits themselves are not useful for teaching learning.

Table 2.2.1: Summary of Literature Review

Reference	Focus Area	Key Contribution	Limitation
[1], [3], [10]	Logic Simulation Tools	Development of logic simulators (gate-level) for designing and testing digital circuits.	Limited to logic gates; no CMOS transistor-level visualization.
[2], [7]	SPICE & Transistor-Level Simulation	Accurate modeling of MOSFET behavior, circuit performance, and switching characteristics.	Complex for beginners; lacks intuitive visualization.

[4], [5]	Boolean Expression Processing	Parsing and simplification techniques (K-map, algorithms) for optimized logic design.	Focuses only on logic optimization; no circuit or layout generation.
[6], [8]	CMOS Circuit Design	Implementation of pull-up and pull-down networks using PMOS and NMOS transistors.	Does not include visualization or interactive tools.
[9], [21]	Layout Design Tools	Physical layout generation and transistor placement techniques.	Requires expertise; not beginner-friendly.
[11], [23]	Web-Based Visualization	Use of web technologies (Flask, JavaScript, D3.js) for interactive systems.	Limited integration with CMOS circuit generation.
[12], [17], [19]	Educational Tools	Interactive platforms to improve learning and understanding of digital systems.	Mostly static or gate-level; lacks transistor-level depth.
Proposed System	Integrated CMOS Visualization Platform	Boolean Expression → CMOS Circuit → Layout → Visualization → Simulation in a single web application.	Simplified layout; not industry-level design rules.

2.3 Summary

Existing investigations on Digital logic simulation, CMOS circuit design, Layout generation, and visualization technique have been reviewed in this chapter. Although there has been much progress in each area, most work has concentrated on individual components and there is very little that considers the question of an integrated approach. There are no logic simulators that include transistor-level details and circuit simulators that are easy to learn but don't provide as much high-level understanding. Equipment to create a layout includes equipment to design the physical layout but does not have accessibility and needs specialized knowledge and experience.

The review reveals an obvious lack of the interactive integration of Boolean logic

processing, CMOS circuit generation, circuit layout visualization and real-time simulation. To fill this gap, this project aims at creating a web application to visualize and generate CMOS Logic Circuits, comprising all of these aspects. The system proposed is intuitive and interactive to learn about CMOS design from high level theory to implementation.

Chapter 3
Software Requirements

The entire development of an Automated CMOS Logic and Layout Visualization Tool are detailed in this chapter. It consists of three main blocks that include the following: logic parsing and synthesis, generation of circuits at the transistor level of CMOS technology, automatic visualization and verification of the circuit layout. The proposed system's goal is to create an open-source Electronic Design Automation (EDA) tool's structure along with integrating a theoretical CMOS logic design and physical design in one unified structure for VLSI. The implementation begins by taking in a Boolean expression or a Verilog description from the user (via graphical or command-line input). These expressions are used to be analysed and converted to structured logical representations using parsing algorithms. The logic synthesis phase then further optimizes the gate level logic, and converts the design into a network of pull-up and pull-down transistors in CMOS. The resultant circuit at the level of transistors is employed for the automatic creation of schematic diagrams, stick diagrams, and structure layout. Lastly, generated layouts are simulated and verified with simulation and verification tools like Ngspice, KLayout. The detailed methodology, implementation architecture, software integration, process of layout generation, simulation framework and visualization techniques used in the project are described below.

The proposed system is implemented completely using software technologies, as it is designed as a web-based visualization and simulation platform. The software required contains user interaction (frontend) and logic processing/circuit generation (backend) technologies.

3.1 System Architecture

The CMOS Logic Circuit Visualizer is designed to be a modular system, with various functional blocks connected together. The overall system workflow consists of a series of steps starting from the user typing a Boolean expression to the Web interface. The user enters an expression and it is sent to the back-end server which parses and processes it. Processed logic is then used to create the pull-up and pull-down transistor networks. The layout generation module generates a simplified layout of the CMOS design after generation of the circuit. Finally, the circuit and layout generated are shown on the front-end interface with simulation outputs.

Architecture provides separation of user interaction, visualization modules for modular and scalable system. This design concept makes the system more maintainable and facilitates adding functionality for generating advanced Layouts.

Figure 3.1.1 represents the system architecture of a web application for a CMOS Logic Circuit Visualizer. It starts at the front-end, where a user types in a Boolean expression in a website, and then presses the generate button. Normally the expression would be passed to the back-end server for processing over an HTTP request. The parsing of the expressions in a Boolean syntax is performed by the expression parser and checked for its correctness; the parsing of the parsing logic to a logic structure is performed by the logic processor and is simplified. The Pull-Up Network (PUN) is then generated by the transistor network generator with PMOS transistor, and the Pull-Down Network (PDN) is generated from NMOS transistor. Logic data for generated circuits can also be entered into what may be a database for saving user histories, circuit templates, etc. Last, the output and visualization section shows the user the CMOS circuit diagram, layout view and simulation waves, respectively. The architecture offers an automated and interactive system for: Boolean logic processing, CMOS circuit generation, visualization of the layout, analysis via simulation.

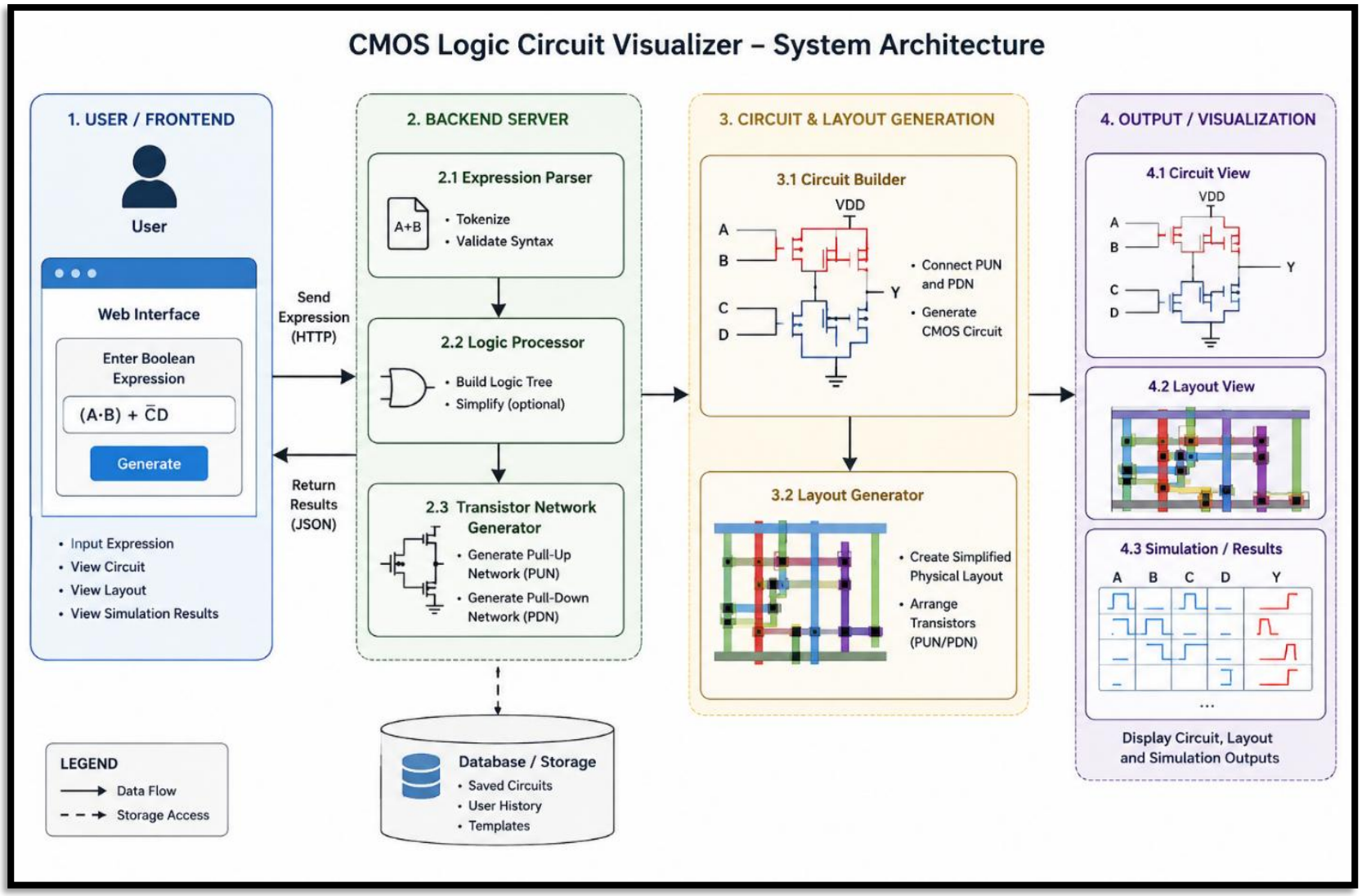


Figure 3.1.1: System Architecture of the software

The architecture of the system is a three-layer web application:

3.1.1 Presentation Layer

It includes user interface, graphical visualization and interaction of the CMOS Logic Circuit Visualizer and Layout Web Application.

Software Technology

HTML5 include blocks the Webpages and application construction.
HTML5 – Builds website structures according to conventions.
TypeScript – Type-safe front-end development and programming
Federer – SMS gateway & management tool for remote internal communications (RIC) SMS gateway & management tool for remote internal communications (RIC)
Opinions – A set of all the elements.
Most of the elements– Opinions is a bundle of all the elements.
For circuit diagrams, stick diagrams that typically appear in CMOS operation analysis, stick diagrams are rendered by using the library D3.js.
These are the two most popular workflows used to develop a website. The two he most popular are the following two workflows:

Functions

1. Take some formula from user.
2. Get a Boolean expression from the user.
3. Create and simulate CMOS transistor level circuits.
4. Create stick diagrams, and circuit layouts in CMOS.
5. Discontinue showing false tables and logic analysis of logic
6. Abilities to develop response and interactive graphical interfaces
7. This technology aims to build an AI-based system that can visually explain Boolean reduction and circuit synthesis.
8. The idea is to create an AI-powered system capable of explaining Boolean reduction/circuit synthesis visually.
9. Error checking and validation of syntax in Realtime as information is inputted.

3.1.2 Application Layer

It controls the processing of Boolean logic, the synthesis of CMOS circuits and serves as a computing engine in between the frontend and visualization modules.

Software Technology

MongoDB – Collection data store that utilizes a NoSQL approach

Post CSS – server that handles posting HTML/CSS.

The language for implementing "backend logic" is "Typescript. Language for implementation of "backend logic" is "TypeScript.

Handling & processing order based on state, activating uses Memo & Effect

Adds support for parsing and simplifying Boolean expressions. Adds support to parsing and Boolean simplification.

The circuit Generator Module function allows you to synthesize CMOS transistor networks. The circuit Generator Module function can be used to synthesize CMOS transistor networks.

AI2RXT EXPLANATION FOR LOGIC SIMPLIFICATION & SYNTHESIS – AI rationale for logic simplification & synthesis using Google Gemini API

Functions

1. Input Boolean expressions into the frontend.
2. Input Boolean expressions to the front end.
3. Convert expressions to an Abstract Syntax Tree (AST).
4. Construct truth tables for Logic Evaluation problems
5. Find the minimal SOP of a given Boolean function using Quine-McCluskey algorithm
6. Using critical pairs for finding monotonicity conditions on CMOS synthesis
7. Manufacture and characterize switching transistors for circuits: transistor diode and transistor NAND gates.
8. Fabricate and measure the performance of Switching transistors (transistor diode and transistor NAND gates).

9. The restriction on using internal nodes for CMOS transistors needs to be relaxed. Relax the limitation on using internal nodes in CMOS transistors.
10. Install inverter stages as needed.
11. Load data for visualization in synthesized circuits

3.1.3 Data Layer

It controls the store, setup and relay of circuit data, and the circuit-generated synthesis data in the application.

Software Technology

Store intermediate data structures: Circuit objects and arrays

React State Management: Provides a way to keep both data on the front-end and back-end synchronized.

Mutation of Obstructions – Used for the modifications of circuit or transistor information.

Optional MongoDB / MySQL – May be added later on for further project storage in the future.

Functions

1. Write Boolean expressions and simplified equations.
2. Operates a truth table.
3. Works with truth table information.
4. The structures of the transistors are saved.
5. Transistor network structures are stored.
6. Map node to node and store circuit configurations
7. Data transfer of translator to visualization modules.
8. Create referential tables for/stick diagram data.
9. Provide for organized communication between the front-end and back-end layers.

3.2 Frontend Software

The user interaction, graphical visualization and real time display of CMOS circuit synthesis results are done by the front-end software of the CMOS Logic Circuit Visualizer & Layout Web Application. Built with cutting-edge modern web practices using React, TypeScript, Tailwind CSS, D3.js, and Vite. The frontend offers an interactive and responsive interface for users to input the Boolean expression and to visualize the CMOS circuit and stick diagrams at the level of the transistors on the fly.

The main frontend component is App.tsx that serves as the root component of the application. Supervises the input from users, validation of expressions, producing output, and interaction with the back-end logic and visualization components. The application keeps a close watch on the changes the user makes to the Boolean expression and activates real-time parsing/synthesis operations. The component is also able to detect errors and will show popping errors who are not supported or should not be in the text (e.g. Unbalanced parenthesis, Unrecognized variable, etc.).

The frontend visualization system's two main pieces of code are CircuitDiagram.tsx and StickDiagram.tsx. The CircuitDiagram.tsx component is built using D3.js to display transistor-level CMOS circuit diagram. It depicts Pull-Up Networks (PUN) and Pull-Down Networks (PDN), VDD and GND rails, inverter stages, transistor interconnections and output nodes. The positioning of the transistors and network layout are determined dynamically in this rendering system, according to the size of the synthesized Boolean (logic) function. The working of Recursive layout algorithms is used to arrange an efficient arrangement of series and parallel Transistor configuration.

The StickDiagram.tsx is a component used to create physical-level VLSI stick diagrams – that is, CMOS layout layers. It shows the N-Wells regions, P-Substrate areas, polysilicon gates, diffusions, metal connectors and contact vias in colorful graphical elements. The stick drawing of CMOS implementation gives the user a simplified overview of the physical implementation and gives insight in abstraction at the layout level when implementing a VLSI circuit.

The Frontend is styled with Tailwind CSS, and is responsive. The application has user-friendly and professional technical interface, inspired by CAD-Tool. The layout consists of separate sections for input handling, logic analysis, truth table generation, visualization of circuits and visualizing their stick diagram. Responsive grid layouts work automatically on all screens.

The application entry point is the main.tsx which is used to start React and render the root application component at the HTML document. The index.html file serves as a template for all HTML, and for React, it's the root container. In order to manage global styling, an index.css file is provided to import the Tailwind CSS directives and set the styling that is consistent throughout the front-end components.

The front end also uses the Google GenAI SDK to integrate the Gemini AI API. Once backend has finished Boolean simplification and circuit synthesis, the frontend request async explanations from backend, using AI features indicating what has been simplified and synthesized. This functionality would add to the merit of the application from the educational point of view as it gives detailed description of the Boolean reduction/ CMOS implementation steps.

3.2.1 Frontend Features

1. Boolean Expression Entry Box

Users can input expressions such as:

F=AB+C

F=(A+C) (B+C)

2. Interactive UI

On-the-fly updates

Dynamic page rendering

User error messages

3. Responsive Web Design

Operates on:

Desktop, Tablet, Phone, Laptop

4. Graphical Representation

Illustrates:

CMOS circuits

Truth Table

Stick Diagram

3.3 Backend Software

The computational algorithms for logic simplification, truth tables, computing Boolean expressions, and synthesizing CMOS transistor networks are implemented in the backend software of the CMOS Logic Circuit Visualizer and Layout Web Application. The backend is fully written in TypeScript and runs synchronously, in the context of a React-based application architecture.

Boolean algebra engine is implemented in the central backend module, called booleanEngine.ts. This module will take a Boolean expression from the user and parse, evaluate, simplify and provide logic analysis of the expression. Using recursive descent parsing, the parser translates a Boolean expression written in textual format to an Abstract Syntax Tree (AST) representation. The parser can recognize variables, logical operators, Boolean constants, and parentheses.

The rules of operator precedence are applied in a number of stages of parsing. The highest priority operations are the OR operations, followed by the AND operations and the NOT operations. The parser recursively builds the tree structure correspondence between a variable and an operator, and the logical relationship. These AST structures are then utilized all through the synthesis process.

The backend also produces recursive evaluation techniques to produce complete truth tables of Boolean functions. evaluate will evaluate truth values for every possible combination of truth values of the function's arguments, and get Truth Table can create truth tables for Boolean functions of up to three arguments. The back end also checks if functions are positive monotonic or not, thereby helping to develop efficient CMOS synthesis strategies.

The Quine-McCluskey algorithm is used to implement Boolean simplification. Simplification process includes identification of min terms from the truth table, grouping

of implicants by their bits, combining like with like and finally extracting the essential implicants. The easier-to-understand Boolean expression is then recreated as a minimized Sum-of-Products (SOP) format. The simplification steps are also given right to the human eye to make it easily understandable.

The second large backend module is circuitGenerator.ts that takes simplified Boolean expressions to generate CMOS transistor level circuits. This module takes a Boolean tree as input and generates complementary CMOS Pull-Up Network (PUN) and Pull-Down Network (PDN). Boolean logic is directly implemented when designing NMOS transistor networks and is applied using De Morgan's duality principles when designing PMOS network transistors.

The synthesis engine has a recursive structure that creates transistor networks from three logic block structures: transistor blocks, series blocks and parallel blocks. AND operations are mapped as series transistor connection and OR operations are mapped as parallel. All intermediate nodes are generated by the system automatically for series connections and suitable source and drain ports are allocated automatically to each transistor.

In addition, the backend decides if further stages of the inverter are needed in the light of monotonicity analysis. Inverter transistors are automatically included in the synthesized network if the synthesized network gives the complement of the required function; the synthesized function is called a sum Circuit. If the synthesized network can generate the complement of the required function, inverter transistors are automatically inserted to give the correct final output; the synthesized function is called a sum Circuit. This system then boils down all the structures of the transistors into arrays.

Constant conditions (always-outputs) are also supported with special handling. In such cases, simplified symbolic representations are created without unnecessary structures of transistors.

I. Request Processing - Handles request from frontend.
II. Expression Parsing - Transmits Boolean expressions to parser engine.
III. Circuit Construction - Develops CMOS pull-up/pull-down circuits.
IV. Simulation Management - Starts simulation programs.

3.3 Expression parser

The Expression Parser will convert the Boolean expression to the internal structured representation. The parser examines the expression and creates a syntax tree that will form the hierarchical structure of logical operators and variables. The parsing process is to find out logical operations like:

- AND operations
- OR operations
- NOT

operations the parser breaks down the input expression into smaller components and organizes them into nodes of the syntax tree. This structured representation can be efficiently processed and transformed during the CMOS circuit generation. The syntax tree constructed by the parser is the basis of the logic processing stage. It also guarantees that complicated Boolean expressions are correctly and systematically understood.

3.4 Logic processing engine

Figure 3.5.1 presents the Logic Processing Engine (LPE) design of the CMOS Logic Circuit Visualizer (LCV) system. This module takes the Boolean expression and translates it into a simplified Boolean expression and validates the precise meaning of the expression. The input to the process comes from the parser, where the parser receives the Boolean expression and analyses it. The system at the Expression Analysis stage starts with an analysis of the expression in question, breaking it up into a set of tokens, constructing an expression tree and recognizing variables and operators.

Then the Simplification Engine reduces the expression to minimize it and to eliminate redundancy according to the rules of Boolean algebra and De Morgan. Validation and Consistency Check stage checks validity of syntax, usage of variables, inconsistency and does a logical consistency check after a simplification process. In the next stage (Preparation for CMOS Implementation), processed expression goes to the CMOS implementation of the expression and the expression is converted to the forms suitable for CMOS design like SOP/POG form and separate data is prepared for PMOS Pull-Up Network (PUN) and NMOS Pull-Down Network (PDN). The Boolean algebra

rules, truth tables, simplification algorithms and variable dictionaries are hypertexts that execute internal processing as part of performing a task. Lastly, the module generates a simplified Boolean expression and transistor network data for additional circuit and layout development. Based on the CMOS design principles, the CMOS generation module generates accurate transistor level Pull-Up Networks (PUN) and Pull-Down Networks (PDN).

Post-simplification, the next step is the Validation and Consistency Check. At this stage, the engine evaluates the syntax consistency of the Boolean expression and validates the appropriate utilization of all symbols, variables, and operators used. It identifies invalid symbols, missing matching parentheses, unmatched operators, missing operands, and contradictory logical expressions such as A. Errors detected at this stage are communicated back to the frontend UI enabling the user to correct any inaccuracies in the Boolean expression before proceeding to circuit generation. Such validation serves to improve the reliability and accuracy of CMOS circuits produced from it.

Next, the Prepare for CMOS Implementation stage prepares the simplified Boolean expression for conversion into a form suitable for implementation using CMOS. Due to the fact that complementary Pull-Up Networks (PUN) and Pull-Down Networks (PDN) are needed for CMOS circuitry, the engine generates two forms of Boolean expression; original and complemented versions of the same expression. Depending on the requirement during circuit generation, the expression is either normalized to SOP or POS forms. The engine identifies all literals, product terms, and groups logic structure that will form the building blocks of the transistor circuit.

The logic processing engine outputs are then directed to the PMOS Pull-Up Network and NMOS Pull-Down Network generators respectively. The PMOS generator creates the PUN from the Boolean logic expression which is linked to VDD while NMOS generator produces the PDN which is linked to GND. All required transistors connections in series/parallel relationships based on Boolean operations are provided by the logic engine. The frontend visualization module utilizes such information to generate transistor-level CMOS circuits and stick diagram using SVG graphics.

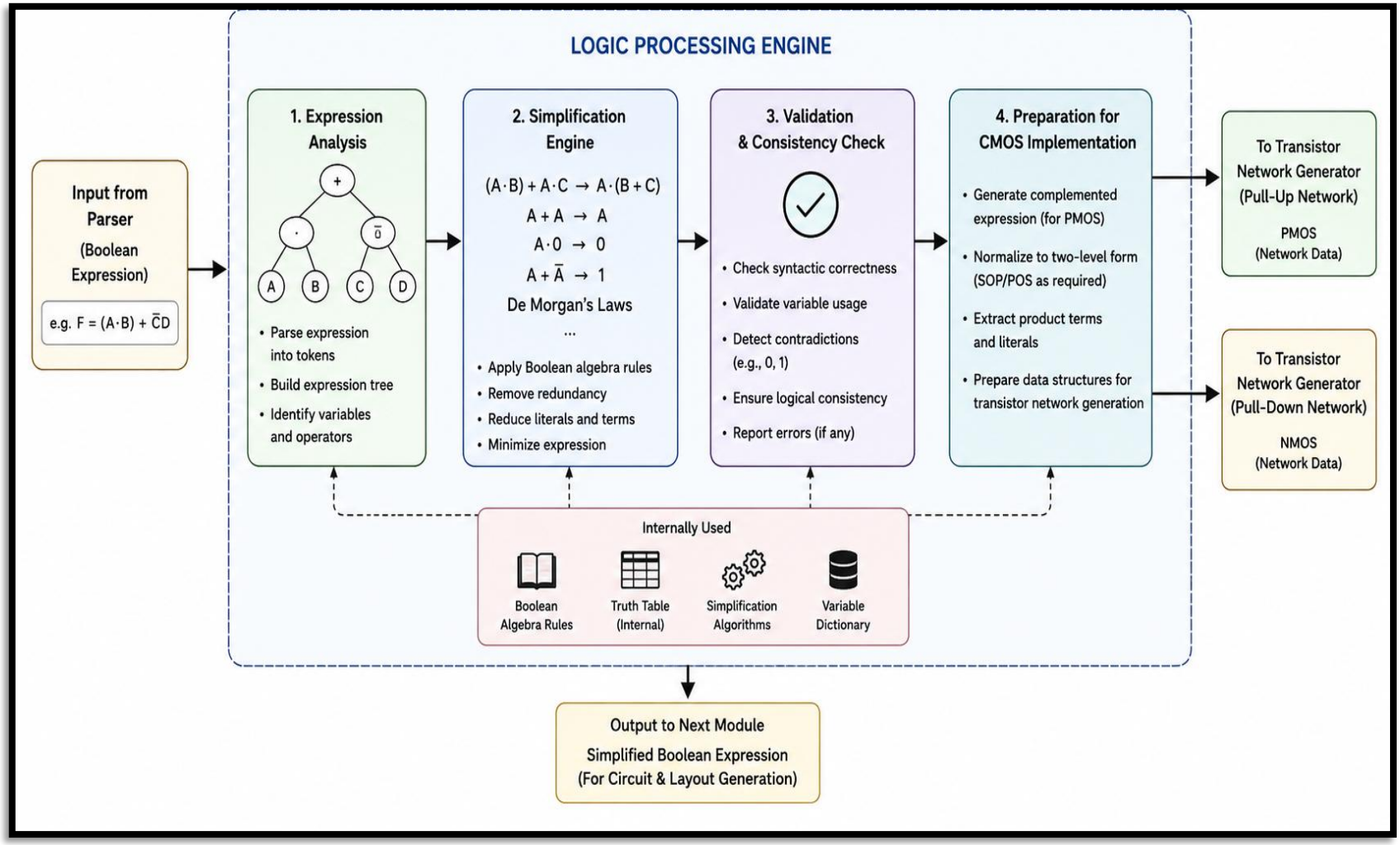


Figure 3.4.1: Block diagram of logic processing engine

3.5 CMOS circuit generator

The main module of the proposed system is the CMOS Circuit Generator module. This module transforms the Boolean expression after processing into transistor level CMOS circuits by generating:

- Pull-Down Network (PDN) with NMOS transistors
- Pull-Up Network (PUN) with PMOS transistors

The pull-down network is used to directly implement the Boolean expression with NMOS transistors. Series connections are AND operations, parallel connections are OR operations. A pull-up network is used to form the transpose of the Boolean expression with PMOS transistors. Here, the series and parallel configurations are alternated depending on CMOS design principles.

The resulting transistor-level structure is a full implementation of a CMOS logic gate. The module also holds information about the connectivity of the transistors and present it for visualization and simulation.

3.6 Layout generator

The Layout Generator is used to generate a simplified physical layout representation of the transistor-level circuit. This is the layout with transistor placement, interconnections and basic routing. The module presents PMOS transistors in the top part of the layout and NMOS transistors in the bottom part of the layout, in accordance with the standard layout in CMOS. The simplified metal routing structures are used to represent interconnections between transistors.

The layout created is not for implementation at the fabrication level, but rather an educational visualization tool to help the user understand the logical design relationship to physical arrangement. This process of layout generation makes concept understanding much better, as it allows to see how the transistor networks are physically arranged in the integrated circuits.

3.7 Visualization module

The Visualization Module is used to show the generated CMOS circuit and the layout structure on the web interface. The module draws the symbols of the transistors, interconnections and layout using graphic rendering techniques like SVG and Canvas. There are various colors and labels to differentiate between PMOS transistors and NMOS transistors. The visualization is interactive and one can dynamically update it as one change the Boolean expression. The module extends the value of the system in the educational realm, allowing users to see the implementation and layout organization of the transistors.

3.8 Simulation module

The generated CMOS circuit is then tested in the Simulation Module that calculates the logic value of the output based on the inputs specified by the user. The module imitates switching characteristics of transistors and determines the active conduction paths. The module updates the logic output as the user changes the input values in real time and the visualization is updated. Active transistors and signal paths are emphasized to assist in understanding the operation of the circuit. The simulation capability allows the user to test the correctness of the CMOS circuit that was generated and to watch the dynamic behavior of the transistor networks.

3.9 Frontend and backend integration

Front and back integration is achieved by using a service tightly coupled to the back server, having a frontend based on React, that directly calls the logic server components with TypeScript imports and uses React state management.

This process starts with user input of a Boolean expression in the input field available in the Frontend. In App.tsx component, the input data is captured and stored in the react state variable. Every time the expression changes, reacts use Memo () hook launches the processing pipeline in the backend.

The frontend calls the booleanEngine.parse method which is available in booleanEngine.ts, before compiling the Bool to an Abstract Syntax Tree. On parsing

exceptions, exceptions are thrown and are caught by the frontend error-management system. If parsing is successful, then the backend will generate the truth table and use the Quine-McCluskey algorithm to simplify the Boolean expression.

To synthesize the CMOS transistor network, the frontend calls CircuitGenerator.generate() from circuitGenerator.ts after simplifying the circuit. The Circuit Object generated includes Pull-Up Networks, Pull-Down Networks, transistor lists, intermediate nodes, information about inverters and the final output equations.

This created circuit data is then forwarded as 'props' to the front-end visualization components. Transistor-level CMOS diagrams are generated from transistor arrays and structures of logic blocks sent to CircuitDiagram.tsx via a d3.js library. At the same time, StickDiagram.tsx uses the same data on a transistor for the stick diagrams that represent the layers of a CMOS physical realization.

The frontend also calls the asynchronous part of the Gemini AI explanation generation, which is achieved by utilizing the useEffect () hook of React. On changes to backend results, frontend builds prompt with simplification steps and sends them to the Gemini API. These explanations are shown in conjunction with the synthesised circuit visualisations.

This application is done by a central state management approach, which is storing all the results generated in a server in the state of the main component. This guarantees consistency for the logic, representation and AI explanations.

This is an architecture for integration which is based on a full processing pipeline:

From that point, the Boolean expression is used to generate a truth table, simplified to generate a simple Boolean equation, synthesized into a CMOS circuit, and then, the generated circuit is rendered to a visual drawing for the human user, before an Explanation of the circuit is generated for the user by AI, to inform and support learning.

3.10.1 Integration Steps

Step 1: Input Boolean expression on frontend.

Step 2: Send Boolean expression to backend API through frontend.

Step 3: Process input expression by backend.

Step 4: Generate CMOS circuit and layout.

Step 5: Return output result to frontend.

Step 6: Frontend presents:

- Circuit diagrams
- Layouts
- Truth Table and Reduction Steps

3.10 Summary

The entire implementation of the CMOS Logic Circuit Visualizer and Layout Web Application was covered in this chapter. This is the framework, software requirements and modules for the designing of CMOS circuits. This chapter elaborated on the processes such as entering the Boolean expressions, parsing, logic processing, creating the CMOS circuit diagram, laying out the circuit, visualization method, and the simulation process involved. The chapter also focused on the interfacing between the front-end and back-end parts of the system. The proposed system integrates various functions into one software application for comprehensively understanding CMOS logic circuits.

Chapter 4
Results and Discussions

Implementation results and analysis of the proposed CMOS Logic Circuit Visualizer and Layout Web Application are presented in this chapter. As a result of development, it has been possible to convert Boolean expression into CMOS transistor circuit layout. The results obtained confirm the correct work of the Boolean expression parser, logic processing engine, CMOS circuit generator, layout generator, and simulation module. The visualization process and features of the layout, as well as the possibilities of real-time simulation are considered in this chapter as well.

Based on the experiments carried out, it may be concluded that the developed system is able to form pull-up and pull-down transistor networks according to various Boolean expressions and display them in real time on the front end.

4.1 Boolean Expression Processing

The developed system successfully accepts Boolean expressions entered by the user and processes them through the parser and logic engine. The parser correctly identifies variables and logical operators such as AND, OR, and NOT and converts them into syntax tree structures.

Rules followed while entering the Boolean expression are as follows

1. There can be no other literals than public variables (i.e., A, B, C, D) in the Boolean expression, which denote logic inputs. While entering the expression, only the Boolean operators such as AND, OR and NOT will be supported. Operators may be used symbolically or as keywords (e.g. AND, "." for multiplication, OR "+" for addition and NOT and apostrophe for inversion). Order of operation should be clearly shown with proper parentheses.

2. Each left parenthesis needs to have an end matching the beginning, which will prevent syntax error in parsing. Balanced parentheses can be used to clearly show a group of expressions and the order of operators. Proper spacing should

be used between variables and operators, making it easier to read and to be processed by the expression parser correctly.

3. The Boolean expression does not need to contain at, #, $, %, special characters, and should not be there as they are not supported by the logic engine. To use NOT, only the NOT should be applied to a single variable or grouped expression. Postulates of the form “AND OR” and incompletes of the form “A AND” are not allowed and lead to an illegal Boolean combination and make it impossible to generate CMOS circuits.

4. All Boolean expressions that appear in the system should follow the conventional rules of Boolean algebra so that the parser, logic processing element and CMOS circuit generator do not fail to produce accurate CMOS pull-up and down transistor networks, stick diagrams, or simulation results, should errors occur.

Input expression examples

1. F = A + B
2. F = A * B
3. F = (A + B)
4. F = (A * B) + C
5. F = (A + B) * C
6. F = (A * B) + (B * C)
7. F = A * (B + C)
8. F = (A + C) * (A * B) + (B + C)
9. F = (A * B) + (C * D)
10. F = (A + B) * (C + D)
11. F = ((A * B) + C) * D
12. F = (A + B + C)
13. F = (A * B * C)
14. F = (A + B) * (A + C)
15. F = (A * B) + (A * C) + (B * C)
16. F = ((A + B) * C) + D
17. F = (A * (B + C)) + D

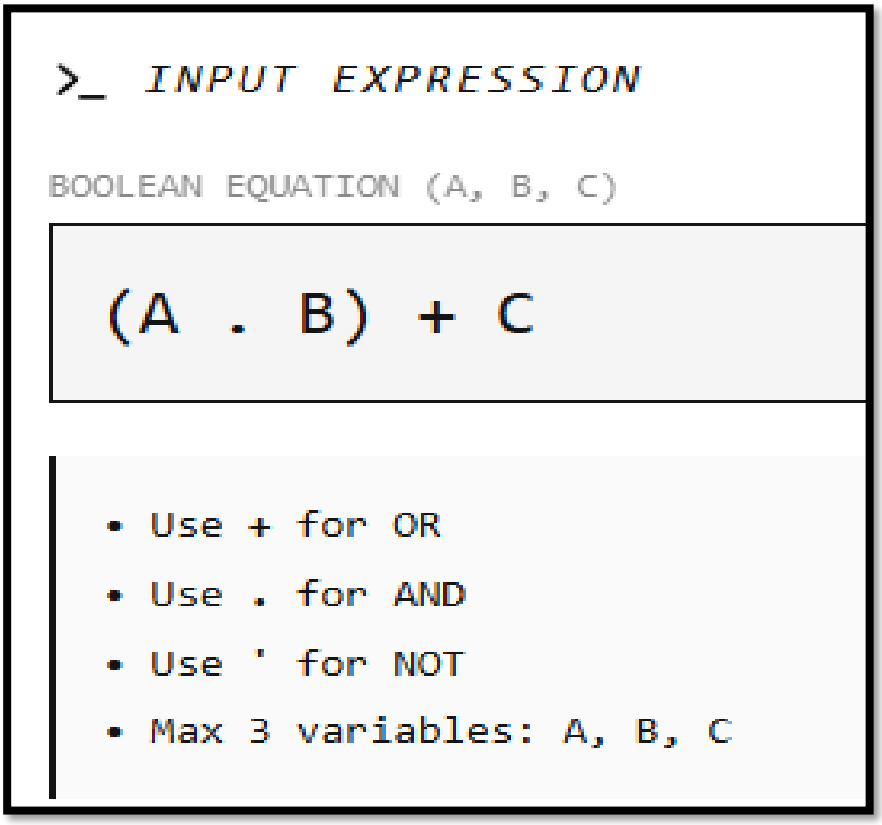


Figure 4.1.1: Boolean expression logic analyzer

Figure 4.1.1 represents the processed expressions are simplified and prepared for CMOS circuit generation. The logic processing engine ensures that the generated transistor networks correctly represent the Boolean logic. BEM (Boolean Expression Input Module) of the CMOS Logic Circuit Visualizer Web App. It offers an easy-to-use interface for the user to enter Boolean expressions with variables A, B and C. Boolean operators such as “+” for “OR” and “•” for “AND” and “’” for “NOT” are accepted in the input box. An expression example is given, (A • B) + C, to illustrate the syntax supported. Figure also shows hints for Inputting a Boolean expression and states that presently, no more than three variables are allowed. After the user is typing the expression, the front end does syntax checking and passes the expression to the back-end parser for logic evaluation, generation of CMOS circuits, layout and simulation. The module provides an easy way to interact with the user and supports accurate handling of Boolean expressions for CMOS visualization.

4.2 CMOS Circuit Generation

The CMOS Circuit Generator successfully generates transistor-level CMOS circuits consisting of:

- Pull-Up Network (PMOS)
- Pull-Down Network (NMOS)

The pull-down network directly implements the Boolean expression using NMOS transistors, while the pull-up network implements the complement logic using PMOS transistors. Figure 4.2.1 represents a CMOS Logic Generator for input expression.

The process of frontend code that arranges the transistor and lines in web application based on Boolean expression provided

The CMOS Logic Circuit Visualizer has an automatic feature to generate CMOS circuits from user entered Boolean expression called the Circuit Generator module. The nutrients are provided by the frontend in the form of web application, created in React.js, TypeScript, SVG and CSS, which dynamically place transistors and interconnection on the web application. Upon typing, the user's Boolean expression is first passed to an expression parser which will reorder the expression into a structured logic tree. The logic processing engine then determines which the Boolean operators are and creates the corresponding Pull-Up Networks (PMOS) and Pull-Down Networks (NMOS). The rendering module in the front end is composed of PMOS and NMOS transistors which are grouped according to CMOS design rules, placing PMOS close to VDD supply line and NMOS close to GND supply line. In an AND operation, NMOS transistors are used in series, and PMOS transistors in parallel. For OR operations parallel PMOS and series NMOS transistors would be used. The front end supports this much by computing the position for a transistor base using coordinate based SVG rendering the transistors, wires, gate connections, sources and drains are all rendered graphically as scalable vector graphics. The connections between transistors are automatically created to create full CMOS circuits. This is connected between the pull-up and the pull-down circuits to create the final CMOS logic circuit. This automatic process of rendering the frontend automates creating clean, scalable and real-time visualizations from the user-provided Boolean expression at a transistor level.

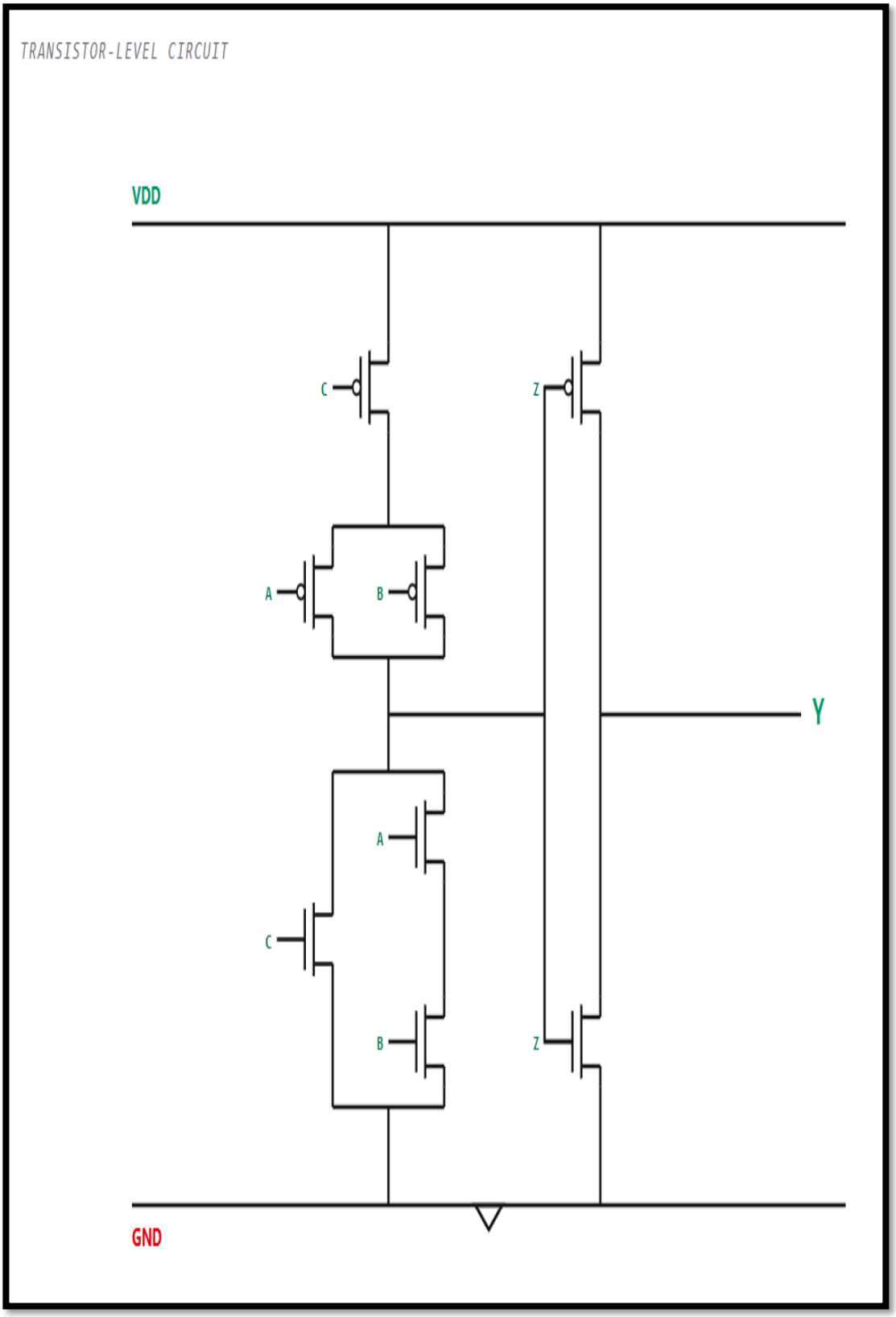


Figure 4.2.1: Transistor level circuit for the input expression

The generated CMOS circuit correctly represents:

- Series NMOS transistors for AND operation
- Parallel NMOS transistors for OR operation
- Complementary PMOS structure

4.3 CMOS Logic Mapping

The generated circuits demonstrate correct CMOS implementation principles. The pull-up and pull-down transistor networks operate complementarily to produce valid logic outputs. The generated structures clearly illustrate transistor-level realization of Boolean logic functions.

Table 4.3.1: CMOS logic mapping working

Boolean Operation	NMOS Network	PMOS Network
AND	SERIES	PARALLEL
OR	PARALLEL	SERIES
NOT	INVERTER	INVERTER

Table 4.3.1 has conceived CMOS circuits follow basic idea of using complementary operation of Pull-Up Network (PMOS) and Pull-Down Network (NMOS). In CMOS design the NMOS will connect to ground as well as the PMOS will connect to the power supply or VDD. Both are designed to generate the correct logic outputs in reverse. Note that for an AND operation, the NMOS transistors are in series to provide a path to ground which all of them must be on, while PMOS transistors are in parallel to provide complementary pull-up activity. The PMOS transistors are mounted in series (series connected) because only if none of them is switched on does no conduction path exist; the NMOS transistors act as series connected with each other, as a complementary structure. If the operation is a NOT, both PMOS and NMOS will create a simple CMOS inverter, i.e. one transistor turns on and the other turns off and gives the NOT result. This complimentary scheme provides for the proper implementation of Boolean logic and low static power consumption while assuring a reliable CMOS circuit operation.

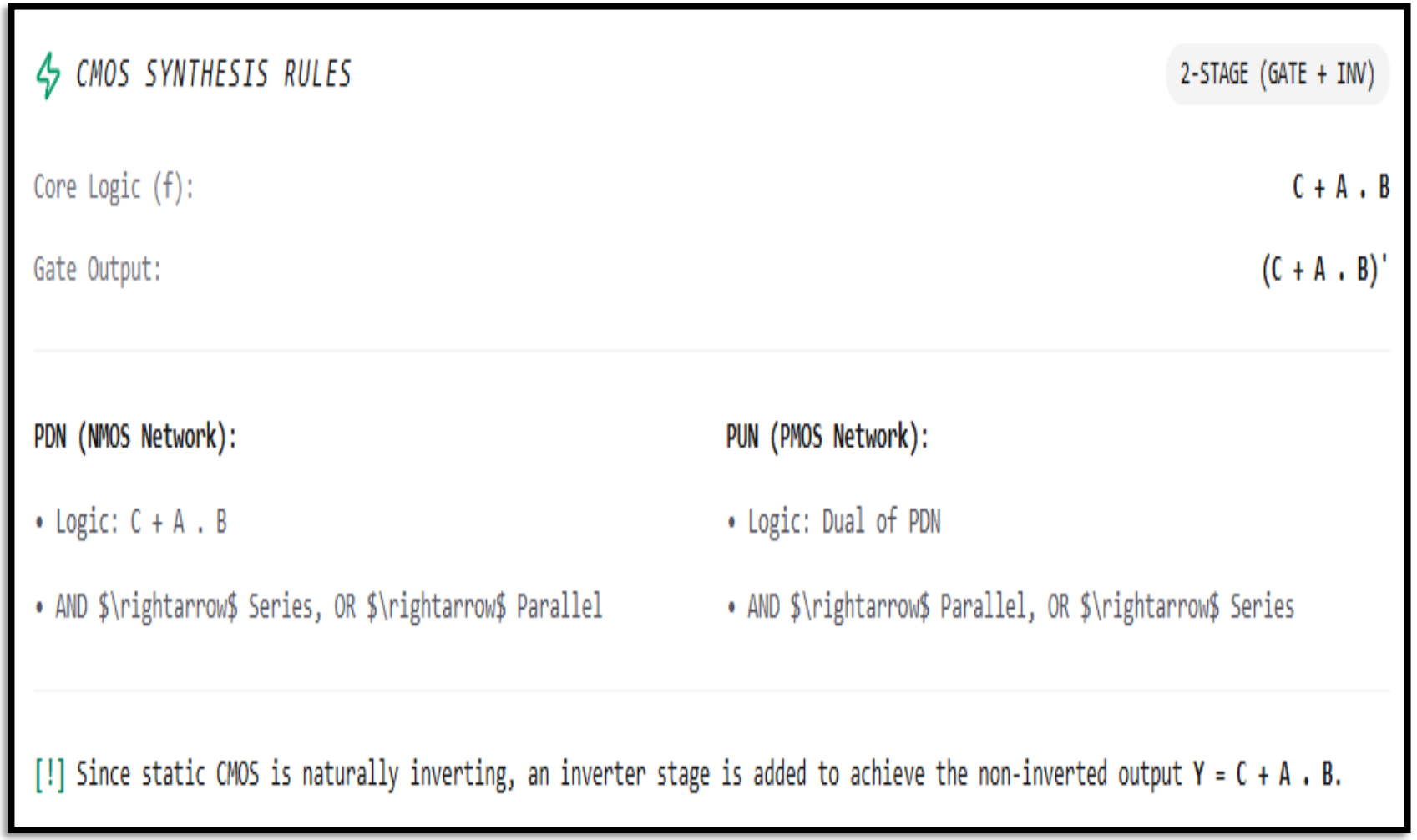


Figure 4.3.1: CMOS logic mapping and rules

The frontier circuit generation module is able to dynamically apply these CMOS mapping rules, rendering the interconnection lines and transistors with SVG. Recursively traverses Boolean expression tree and calculates series/parallel configurations of the transistors. This automated process can be used to create accurate transistor-level CMOS circuits directly from user-defined Boolean functions. The resulting circuits are very accurate in terms of meeting design requirements for CMOS logic circuits; they are also easy to view with a logic education perspective regarding the physical embodiment of the logic functions using complementary PMOS and NMOS transistor circuits.

4.4 Frontend Interface and User Interaction

The developed frontend interface provides an interactive environment for users to enter Boolean expressions and observe generated results. The interface includes:

1. Boolean expression input field
2. Generate button
3. CMOS circuit visualization
4. Layout visualization
5. Simulation output display

The React and TypeScript-based frontend ensures responsive interaction and smooth rendering.

The figure 4.4.1 in the CMOS Logic Circuit Visualizer, the logic analysis stage is shown, where a Boolean expression is analysed and simplified by applying the rules of Boolean algebra. Applying the Commutative Law rewrites the numbers in the original expression so that C is added to a new version of the expression containing only A and B with no multiplication. Simplifying the original expression using the “Commutative Law” creates a new version of the expression with C added to the end and a new expression with only A and B multiplied in place of the original. The system then prints out the steps taken in the reduction along with the resulting reduced Boolean expression. Last, the simplified one is written as the output equation $Y=C+A\cdot B$. This process, which is used to optimize the logic, can be performed prior to the generation of CMOS transistor circuits and layouts for improved circuit implementation.

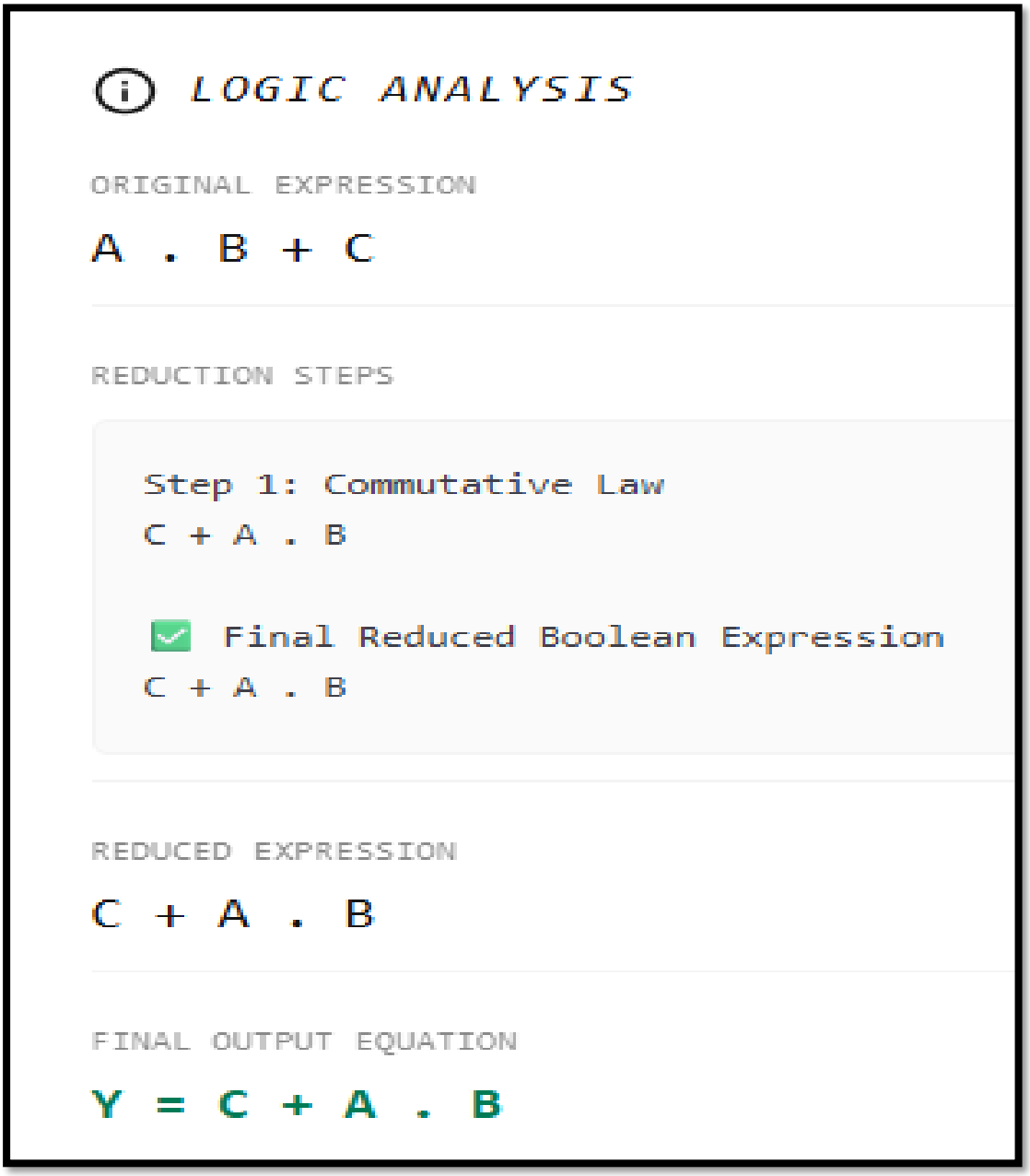


Figure 4.4.1: Logic analysis for the input expression

In figure 4.4.1 The truth table for the Boolean which is created from the expression Y = C + (A * B) is shown above. The list below shows all the combinations of the input variables A, B and C and the resulting output Y. The output is logic 1 if the two inputs are either (A=1, B=1) or (C=1), hence the AND operation A·B. Otherwise the result is 0. The truth table is used to check the correctness of the simplified Boolean expression, and as a logical check of the operation before applying the operation to the CMOS circuit and simulation.

TRUTH TABLE

A	B	C	Y
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

Figure 4.4.2: Truth table and reduced Boolean expression

4.5 CMOS Stick Diagram Visualizer

A simplified diagram of a layout of a CMOS circuit but with the purpose of demonstrating the placement and interconnection of the different levels of fabrication without any exact dimensions. Different colours in a stick diagram stand for different layers like polysilicon, diffusion, metal or contacts. In the typical design, the red vertical lines generally denote polysilicon gates, the blue lines are in the metal-1 layer and are for interconnections, the yellow area is a P-diffusion for PMOS transistors, and the green is

an N-diffusion for NMOS transistors. Black squares represent contacts/vias, which are electrically connected to other layers. This will be the general layout of the transistors with PMOS transistors in the top half of the diagram connected to the VDD power supply and NMOS transistors in the bottom half in the P-substrate attached to ground (GND).

In Figure 4.5.1 the stick diagram is primarily used for layout planning, to check the connection of the circuit, to minimize routing difficulty, to determine the amount of space occupied by the circuit, and to find errors before intelligent routing and final mask layout. Comes in handy when designing CMOS VLSI circuits as it gives a clear idea of the arrangement of the transistors, series and parallel connections, routing paths, and overall structure of the circuit, thereby simplifying the design process and reducing the time required for its development. Its user interface is also very interactive and user friendly, allowing the user to enter Boolean expressions, create circuits and output analysis without the need of elaborate knowledge of VLSI design. System overall performance is acceptable when small and medium sized.

Boolean expressions typed in by the user are automatically translated and drawn by the frontend of the CMOS Logic Circuit Visualizer Web Application into transistors, routing wires and layout layers. All the code of generating stick diagrams is written in React.js, TypeScript, SVG graphics, and coordinate-based rendering. Upon entering a Boolean expression by the user, it will be taken to the parser and logic processing engine of the frontend first. The backend logic modules process the expression and output the structure of the transistor network, which is the Pull-Up Network (PUN) and consists of PMOS transistors, and Pull-Down Network (PDN) and consists of NMOS transistors. This information about the transistor's connectivity is then passed back to the frontend either as an object or in the form of JSON.

This is a logic data used by the frontal rendering engine for the dynamic calculation of the placement of transistors and interconnection lines. The created Stick diagram shows the PMOS transistors inside the N-well region, close to the VDD power highway at the top; and the NMOS transistors inside the P-substrate region, close to the GND power highway at the bottom.

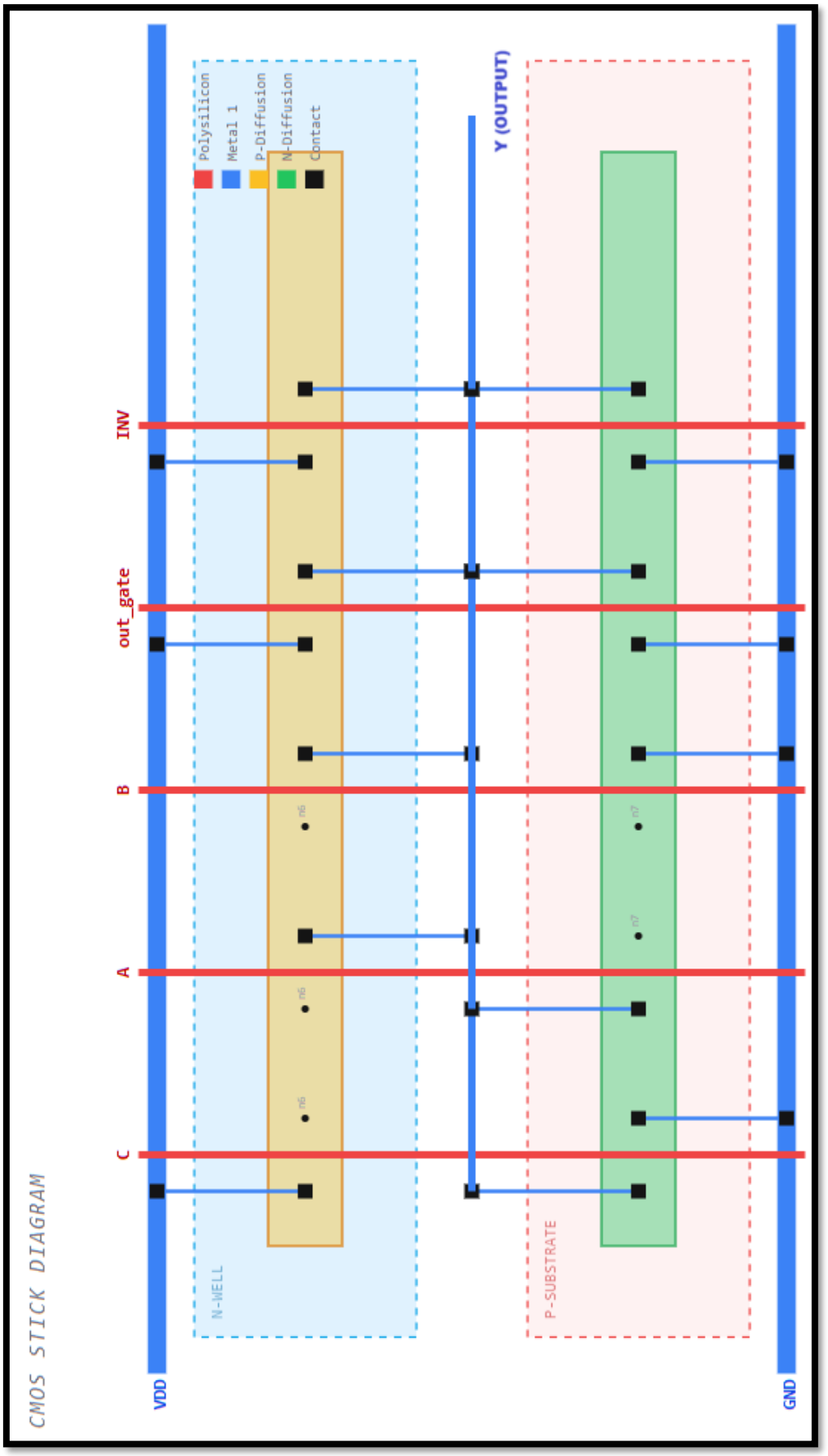


Figure 4.5.1: CMOS stick diagram visualizer

Frontend Features

1. Interactive user interface
2. Real-time updates
3. Dynamic rendering
4. Responsive design

The web interface provides a user-friendly environment suitable for educational and visualization purposes.

4.6 Performance Analysis

The developed system demonstrates efficient performance for Boolean expression processing, CMOS generation, and visualization.

Table 4.6.1 Performance Analysis

Parameter	**Result**
Boolean Parsing	Successful
CMOS Generation	Accurate
Layout Visualization	Dynamic
Simulation Response	Real-time
User Interface	Interactive

The system performs efficiently for small and medium-sized Boolean expressions and provides accurate transistor-level visualization and simulation results. In this table 4.5.1 It is shown that the developed CMOS Logic Circuit Visualizer system exhibits efficient and reliable processing of the Boolean expressions, generation of the CMOS circuits, visualization of the layout and simulation. Based on the performance analysis, the designed system is able to perform all the previous steps from user input to output of the transistor level circuit. The parser gracefully handles the syntactic errors of Boolean expressions and ensures they are composed of valid variables and operators by correctly parsing and validating expression entered by the user during Boolean parsing. The rules of the Boolean algebra are applied efficiently on the simplification engine to produce an optimized logic which can be implemented in CMOS.

The CMOS generation of the module generates precise Pull-Up Networks (PUN) and Pull-Down Networks (PDN) following the CMOS design principles. The resulting circuits accurately implement Boolean logic functions in complementary circuits of PMOS and NMOS transistors. The system also creates simplified physical diagrams to easily show where the transistors should go, how they should be routed, and the structures of the various layers needed for VLSI implementation. Visualization module offers dynamic and interactive graphical output to see CMOS circuits, layouts and logic waveforms in real time. Some features like zooming, layout rendering and plotting the waveforms give the user better understanding of the behaviour of the transistors. The simulation module generates accurate timing waveforms of desired combinations of its inputs very quickly and is used for checking the correctness of the designed logic circuits. The GUI is very interactive and easy to use, allowing the user to enter Boolean expressions, create circuits and output analysis without the need for high level knowledge of VLSI design. In general, the system works effectively with small and medium-sized Boolean expressions, and rightly process the logic, transistors level visualization, and produce simulation results in real-time for the pilot logic without applying to education, research and CMOS logic design.

4.7 Summary

In this chapter, results of the development and detailed analysis of the CMOS Logic Circuit Visualizer and Layout Web Application were discussed. The system was successfully able to evaluate the Boolean expressions, design CMOS circuits on transistor level, plot layout diagrams and conduct dynamic simulations of the logic outputs. The generated pull-up and pull-down network design was able to faithfully reflect the principles of CMOS design, while the visualization and simulation part greatly improved understanding by allowing for a visual interpretation of concepts. Efficient visualization of circuit and layout diagrams was achieved with an efficient frontend user interface.

Chapter 5
Conclusion

The main goal of the project has been effectively achieved via the design and development of a CMOS Logic Circuit Visualizer and Layout Web Application. The goal of the application is to make bitmap the learning interface for understanding CMOS logic design, to be able to transform a Boolean expression into a corresponding transistor- level circuit, and to be able to write a corresponding layout visualization from the transformed circuit automatically.

The newly synthesized system was successful in meeting the criteria of automatic synthesis of either PMOS and NMOS circuits for any Boolean expression in line with conventional CMOS logic design rules as expected. As marked, the system synthesized was able to skip the conventional CMOS design rules to automatically synthesize Pull-Up Networks (PMOS) or Pull-Down Networks (NMOS) for any Boolean expression as expected. Regarding the type of circuits being created, they're all the three simple Boolean operators (AND, OR, NOT). This is achieved by adjusting through complementary transistor sets in the circuits. The visualization system is able to generate the diagrams of the transistor network and the structure of the CMOS layout using D3.js library. These can assist the users in easily picturing the way a particular logical expression is to be represented at the transistor level.

The layout generation module is in charge of the success of the simplified stick diagram versions of the resulting CMOS circuits. This will assist the users to understand the structure and layout techniques used in the VLSI circuits. Finally, the simulation module helps evaluate outputs of the generated circuits by simulating transistor switching behaviour. The developed front-end application is based on React, which makes their interaction and use simple and interactive. It has a well-structured system architecture with modularization that allows it to be easily scalable and maintainable. Finally, this project offers a solution for the important issue faced in education and visualization of logical expression at the level of transistors.

5.1 Merits and Limitations

In this section the principal features and disadvantages of the developed CMOS Logic Circuit Visualizer and Layout Web Application are discussed. The benefits emphasize the benefits of the system in terms of effectiveness for furthering comprehension for the concepts of CMOS design, and the disadvantages point out limitations in the system and potential for enhancements.

Merits

• **Generate the CMOS not only automatically but also as an integrated circuit.**

The system automatically translates a Boolean expression into CMOS transistor-level circuits, which makes the designer more efficient, since manual design of the circuits is more complex.

• **Real-Time Visualization:** The CMOS circuits and layouts generated are dynamically displayed using an interactive web interface which allows users to realize the behaviour of the transistors visually.

• **Educational Learning Platform:** The application is an effective educational tool to students and beginners on CMOS Logic Design and VLSI concepts.

• **Simplified Layout Representation:** It creates the layout structures to explain concepts of placement and routing of physical transistors in a CMOS layout design.

• **Interactive Simulation:** The simulation module checks logic outputs in real time and it shows dynamically the switching behaviour of the transistors.

• **Web-Based Accessibility:** The application is browser based and will not need to download any complex software program making it accessible and easy to make use of.

• **Modular Architecture:** The flexibility of the frontend-backend setup enhances the maintainability, scalability, and extensibility of the system.

• **User-Friendly Interface:** The interface is accessible, smooth, and responsive with React and Typescript. Users have smooth interaction and responsive visualization, thanks to a React and TypeScript-based interface.

Limitations

• **Simplified Layout Design:** The designed layouts are pedagogically useful representations that do not have to meet strict industry fabrication constraints found in professional VLSI design tools.

• **Limited Simulation Accuracy:** The simulation module is mainly a logic evaluation and is not detailed in electrical considerations like propagation delay or power consumption.

• **Scalability Limitations:** The rendering of visualization and layout can be numerically intensive as the circuits are complex.

• **No SPICE Integration:** There is no simulation in the current implementation at the transistor level with SPICE.

5.2 Future Scope

More advanced ideas of VLSI design can be added to the design of the proposed CMOS Logic Circuit Visualizer and Layout Web Application, further increase in the accuracy of the generated layouts and extend the simulation can be performed. An area where improvements should be made in the application is in integrating the use of SPICE to simulate the performance and electrical behaviour of the transistors used. These simulations may include things like delay, energy loss, and switching behaviour. The capabilities for layout generation within the program may be extended by involving industrial design rules, and by incorporating advanced layout generation strategies.

1. Complex combinational circuits
2. Sequential circuits
3. Multiplexers
4. Decoders

Arithmetic Logic Units (ALUs)

The visualization module may be extended to feature:

1. Animated transistor switching
2. Signal propagation visualization
3. Generate Interactive truth table for truth table systems.
4. Timing diagram representation

Automatic circuit optimization and circuit layout generation could be investigated by using machine learning techniques. The proposed system is sound and a good base for further research and development in CMOS circuit visualization, VLSI education and interactive digital design tools.

Appendix A

Supplementary Materials

The information included in this appendix is additional technical information, snippets of software code for implementing the circuits, other explanations, and supporting materials used to implement the project—The CMOS Logic Circuit Generator. The appendix enriches the description of the methodologies, software implementation and CMOS design principles introduced in Chapter 4, 5, and 6. It also serves as a reference guide for any readers interested in digital circuit generation using transistors and seeing the difference between AND, OR logic.

The supplementary materials of our project are verified by manually and in the web interface to check whether the interface is correctly working or not.

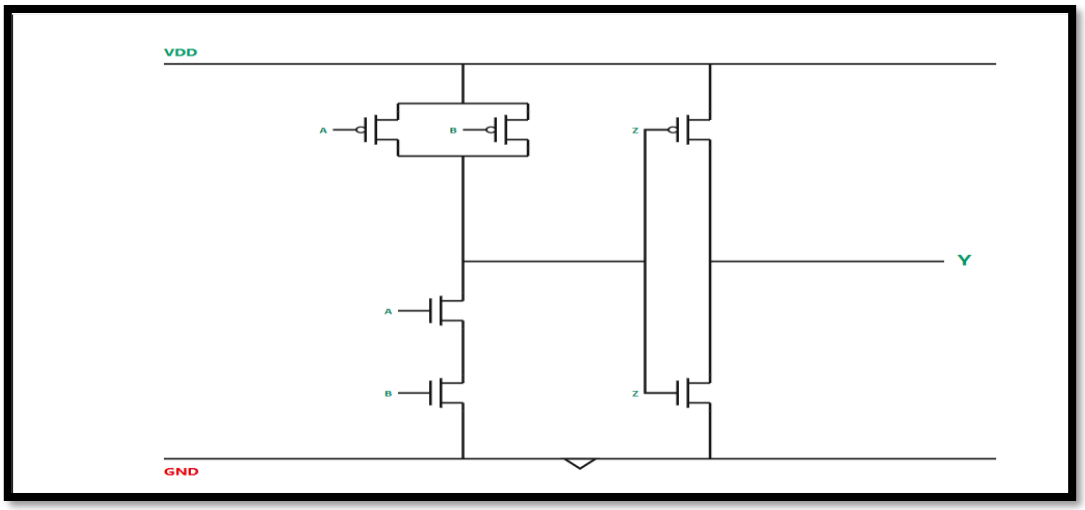
The functionalities and additional features provided by the CMOS Logic Circuit Visualizer were exhaustively tested to validate their proper operation and correctness by employing manual verification techniques and frontend and backend integration tests. The manual verification approach involved testing different Boolean expressions, logical operations, and CMOS generation output separately to test whether the parser, logic processing engine, generation of transistor networks, and layout visualizations produced expected results. Several Boolean expressions that used AND, OR, and NOT operations were input to the system to verify logical correctness by generating corresponding CMOS circuits.

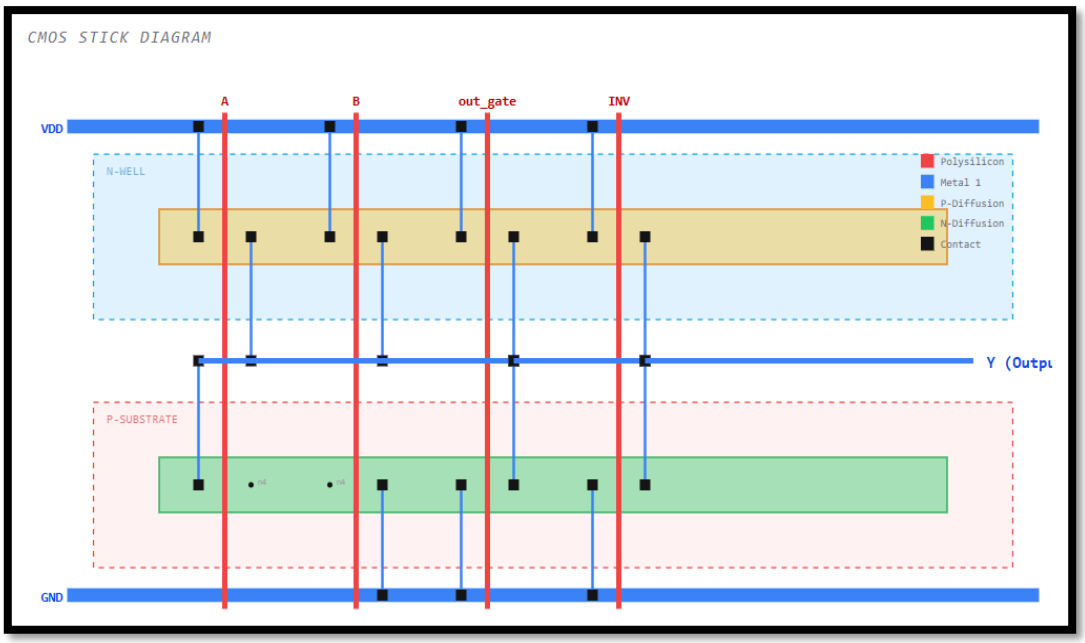
Moreover, the web interface underwent rigorous testing to guarantee efficient communication between frontend and backend systems. The frontend was coded using React.js and TypeScript, and it was tested based on its performance in parsing Boolean expressions, validating them, dynamically rendering output results, and visualizing the generated circuits. The application was analysed using valid and invalid Boolean expressions to test the effectiveness of syntactic validation. Visual examination of the transistors arranged in different PMOS and NMOS transistor structures was performed to test implementation of CMOS logic mapping rules and rules of connecting transistors in series or parallel.

1. **F = ((A.B) + (A.C)'). (A.B) — [DE Morgan's Law, Absorption Law]**

$$F = ((A.B) + (\overline{C.A})).(A.B)$$
$$= ((AB) + (\bar{A} + \bar{C}))(AB)$$
$$= (AABB) + (\bar{A}AB) + (AB\bar{C})$$
$$= AB + AB\bar{C}$$
$$= AB(1 + \bar{C})$$
$$= AB$$

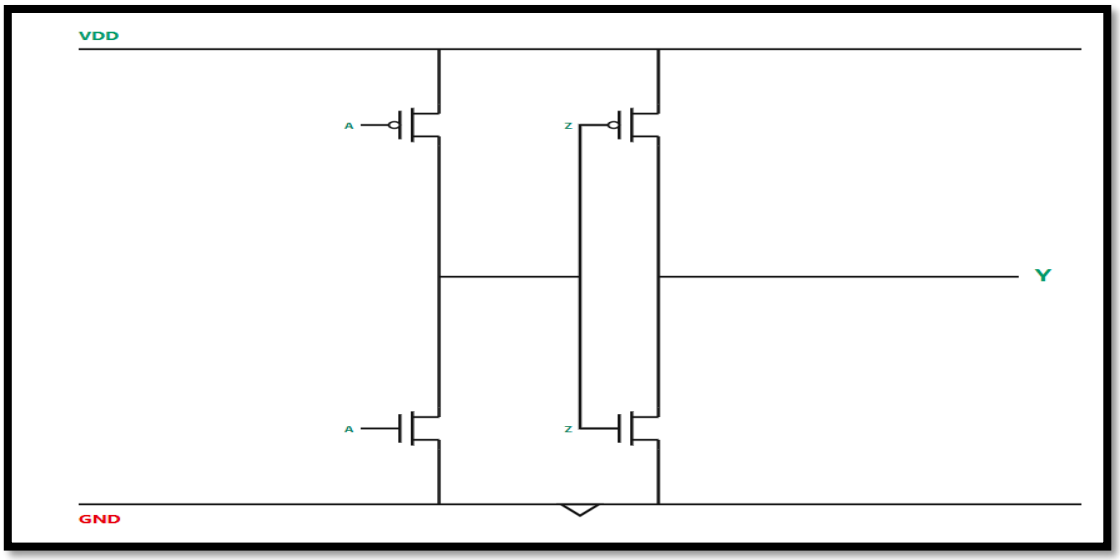
$$F = A\ B$$

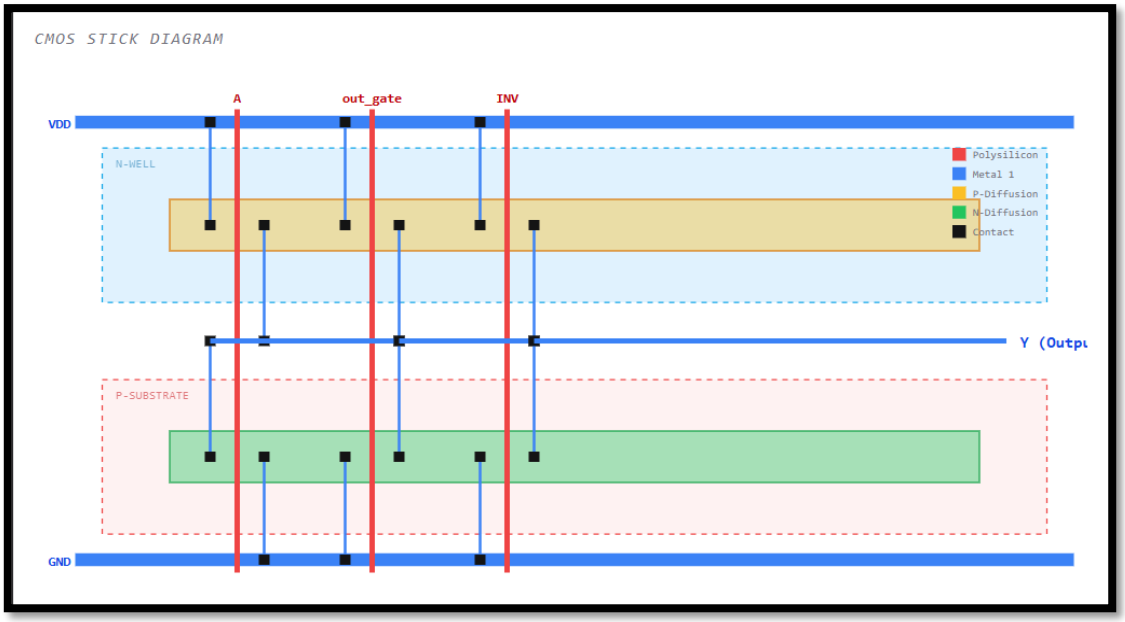




2. F= (A+B.C). (A+B'.C) – [Distributive Law, Law of Union]

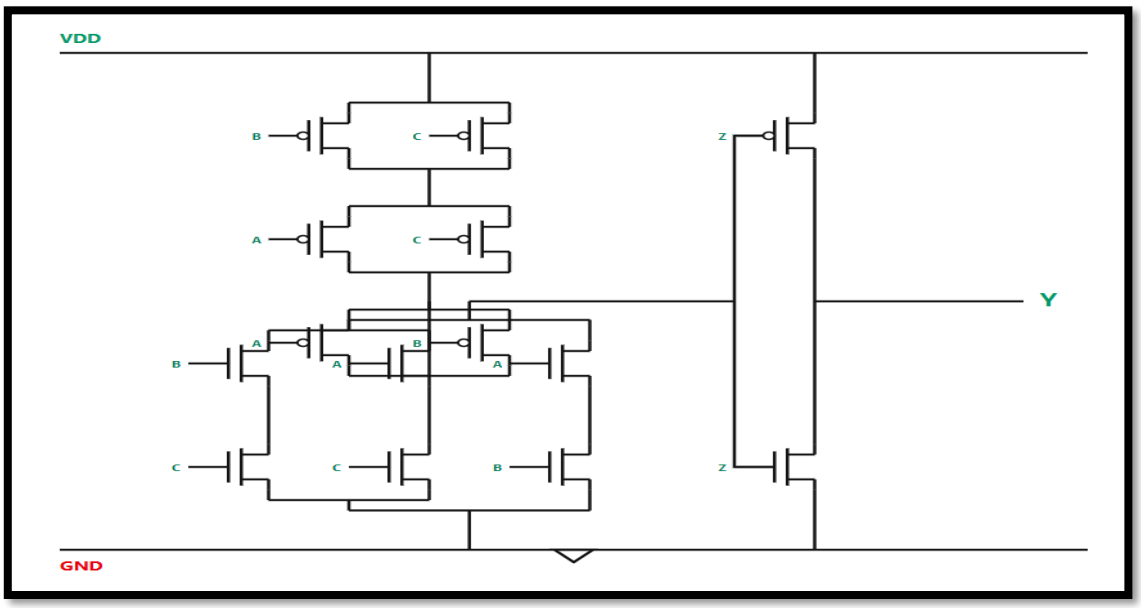
$$F = (A+BC)(A+\bar{B}C)$$
$$= AA + A\bar{B}C + ABC + B\bar{B}C^{(0)}$$
$$= AA + A\bar{B}C + ABC$$
$$= A + AC(\bar{B}+B)$$
$$= A + AC$$
$$= A(1+C)$$
$$= A$$

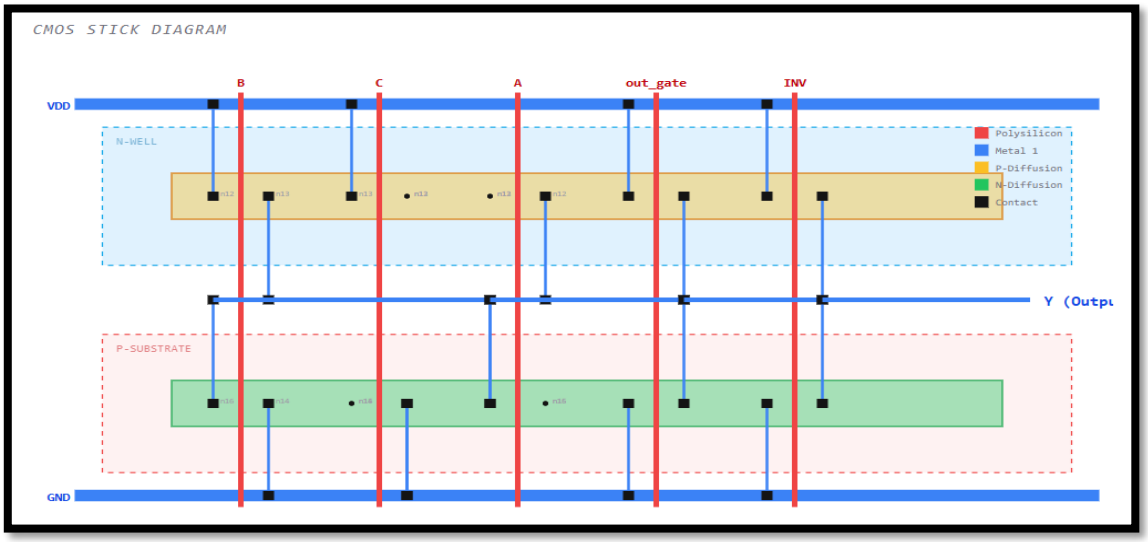




3. **F=((A+B). (A+C). (B+C) – [Law of Union, Distributive Law, Idempotent Law]**

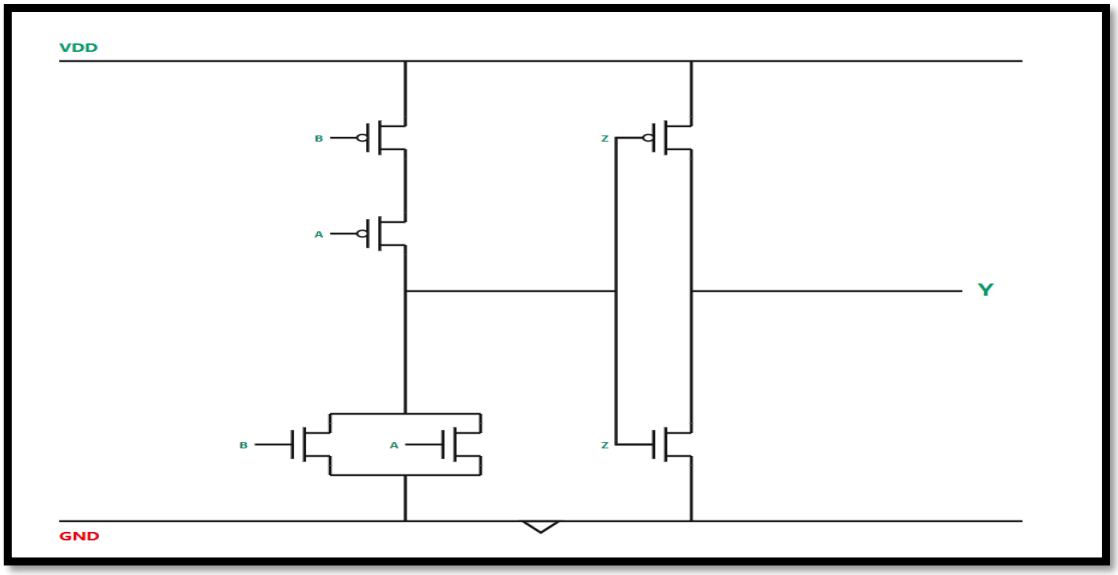
$$F = ((A+B).(A+C)).((B+C))$$
$$= (AA + AC + AB + BC) \cdot (B+C)$$
$$= AAB + AAC + ABC + ACC + ABB + ABC + BBC + BCC$$
$$= AB + AC + ABC + AC + BC + AB$$
$$= AB + AC + BC + ABC$$
$$= AB(1+C) + C(A+B)$$
$$= ABC + AC + AB$$
$$= AC + BC + AB$$

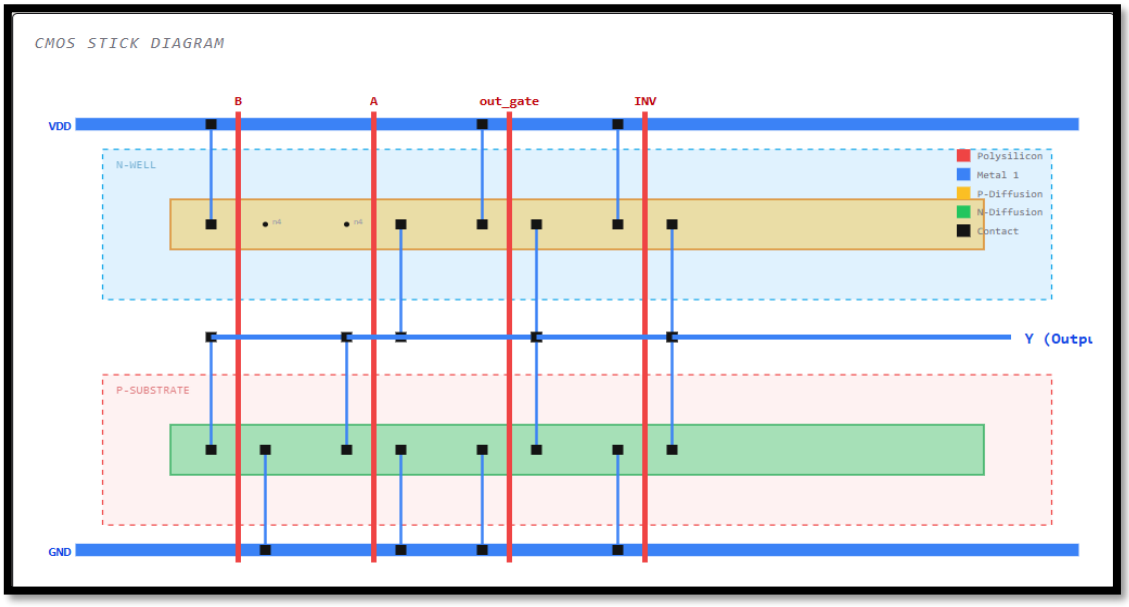




4. **F=(A+B+C). (A+B) – [Law of Union, Distributive Law]**

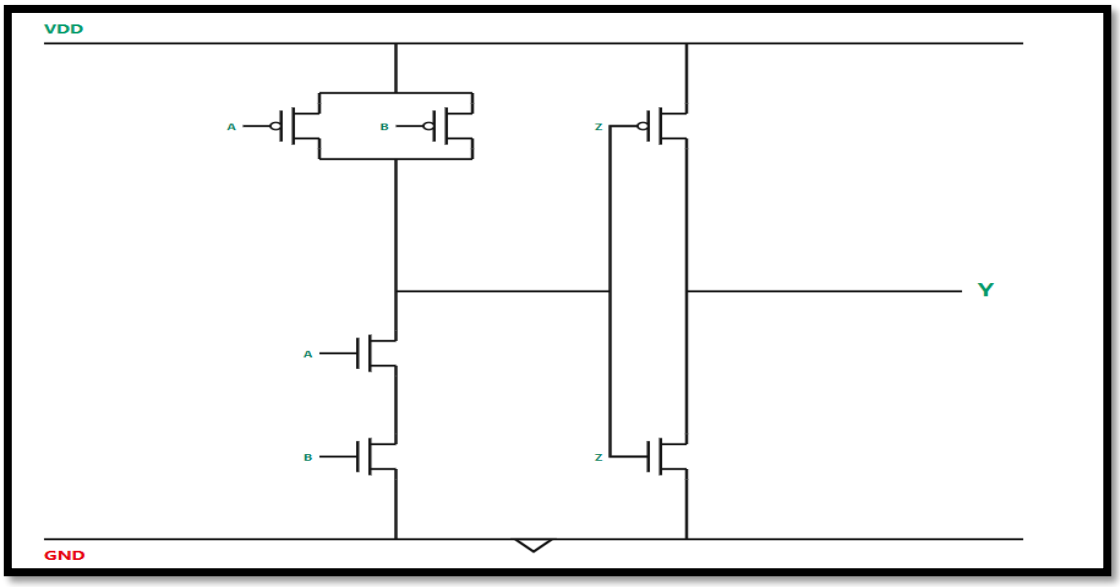
$$F = (A+B+C)(A+B)$$
$$= AA + AB + BC + AC + AB + BB$$
$$= A + AB + B + BC + AC$$
$$= A(1+B) + B(1+C) + AC$$
$$= A + B + AC$$
$$= A(1+C) + B$$
$$= A + B$$

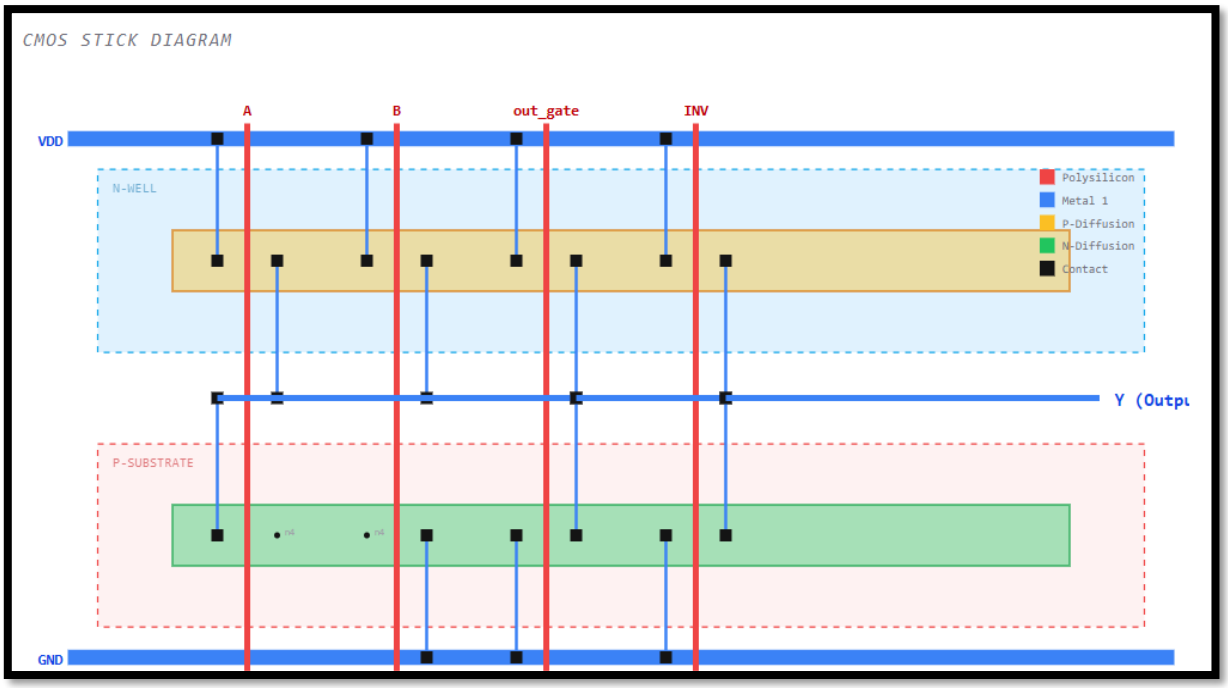




5. F = (A. B+A'. B'.C'). (A.B) – [Idempotent Law, Complementary Law]

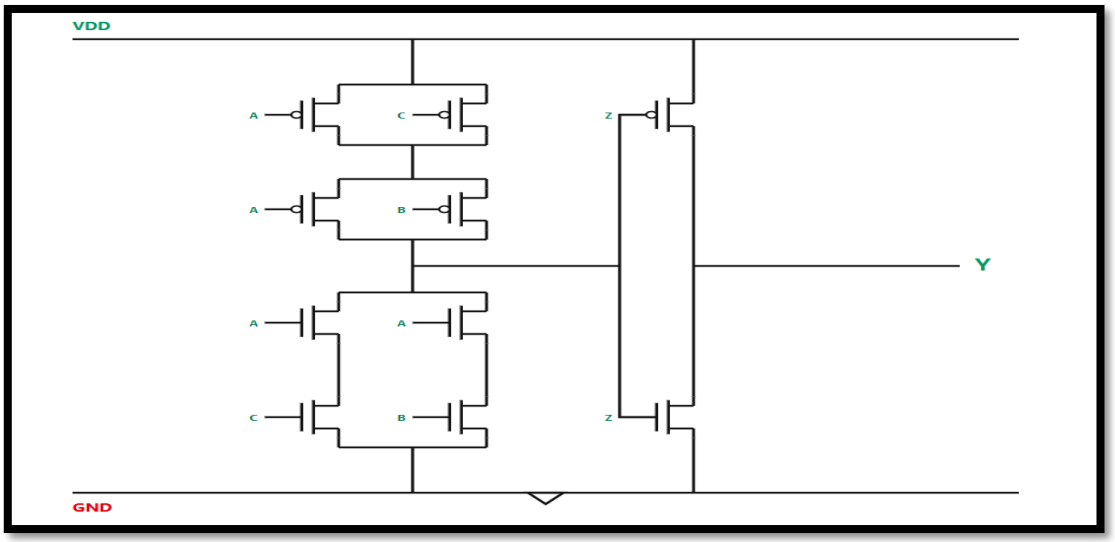
$$F = (AB + \bar{A}\bar{B}\bar{C}) \cdot (AB)$$
$$= AB AB + A\bar{A} B\bar{B}\bar{C}$$
$$= AB + 0$$
$$= AB$$
$$F = AB$$

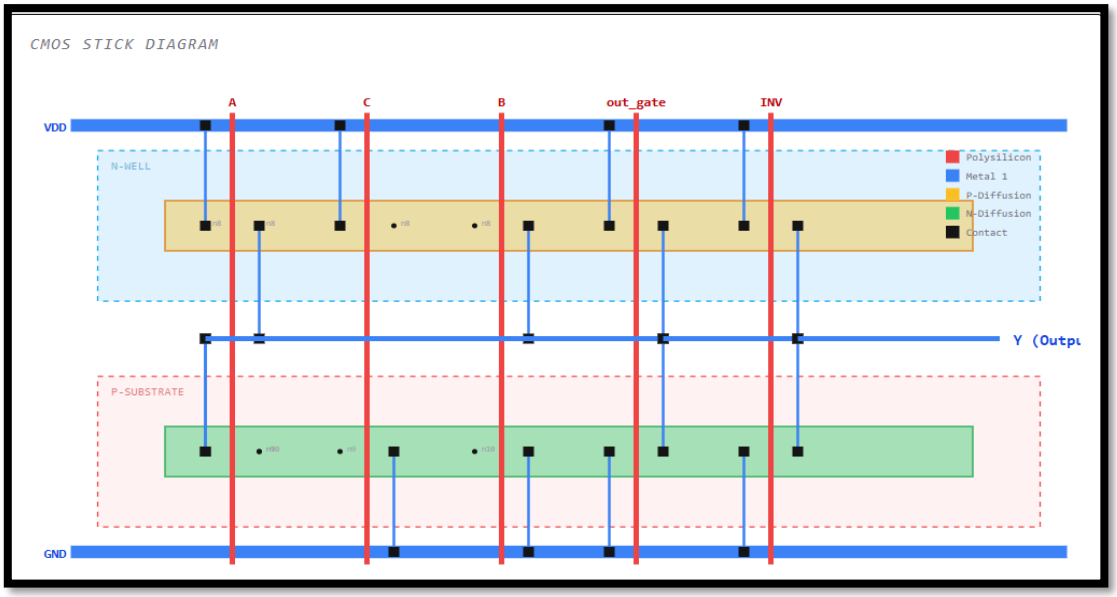




6. **F = ((A. B+A.C). (A. B+C)) – [Idempotent Law, Law of Union, Distributive Law]**

$$F = ((AB + AC)(AB + C))$$
$$= AABB + ABC + AABC + ACC$$
$$= AB + ABC + ABC + AC$$
$$= AB + AC + ABC$$
$$= AB(1 + C) + AC$$
$$= AB + AC$$





7. **F=((A+B'). (A+C). (B+C')) – [Complementary Law, Distributive Law, Law of Union, Idempotent Law]**

$$F = ((A+\bar{B})(A+C)(B+\bar{C}))$$

$$= (AA + AC + A\bar{B} + \bar{B}C)(B+\bar{C})$$

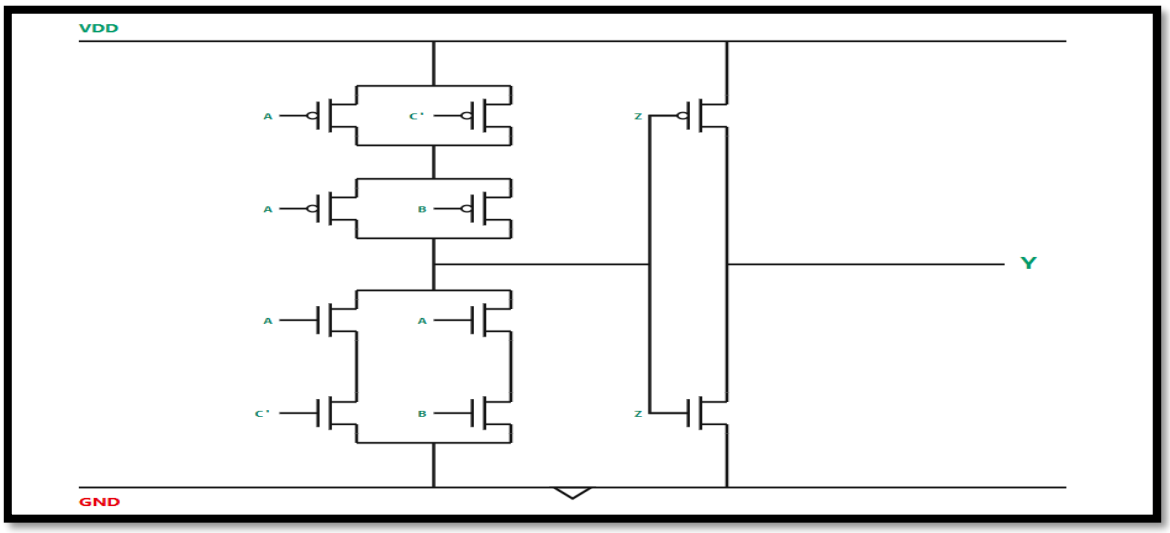
$$= AAB + ACB + A\bar{B}B + B\bar{B}C + AA\bar{C} + AC\bar{C} + A\bar{B}\bar{C} + \bar{B}C\bar{C}$$

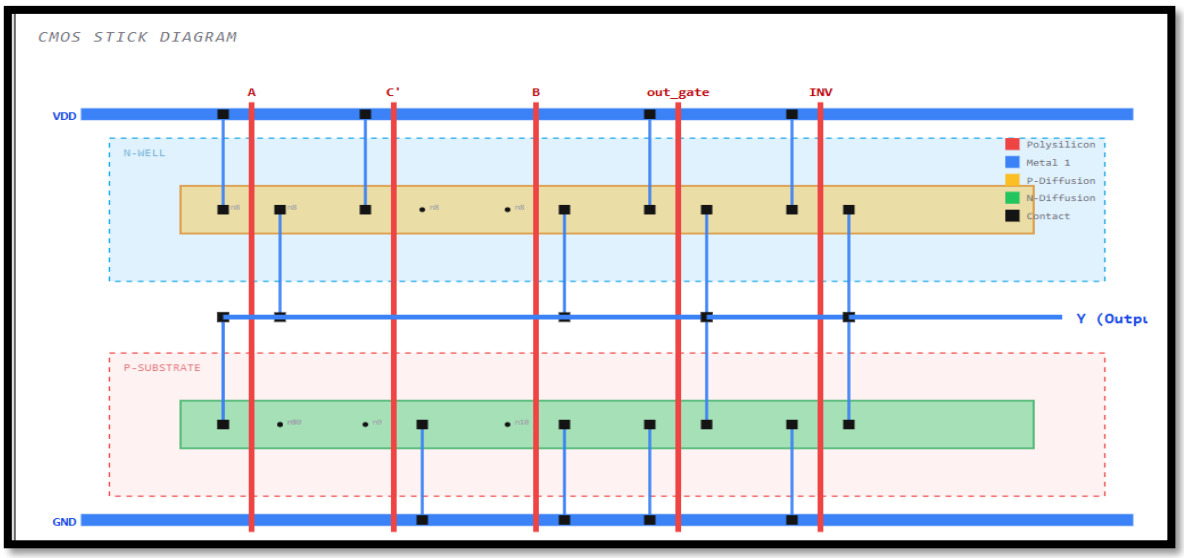
$$= AB + A\bar{C} + ABC + A\bar{B}\bar{C}$$

$$= A\bar{B} + A\bar{C} + \bar{B}\bar{C}$$

$$F = A\bar{B} + AC + \bar{B}\bar{C}$$

$$F = A\cdot\bar{C} + A\cdot B$$





8. **F= ((A+B). (A+C)) + (A.B) – [Distributive Law, Law of Union, Idempotent Law]**

6) $F = ((A+B)(A+C)) + (AB)$

$= AA + AC + AB + BC + AB$

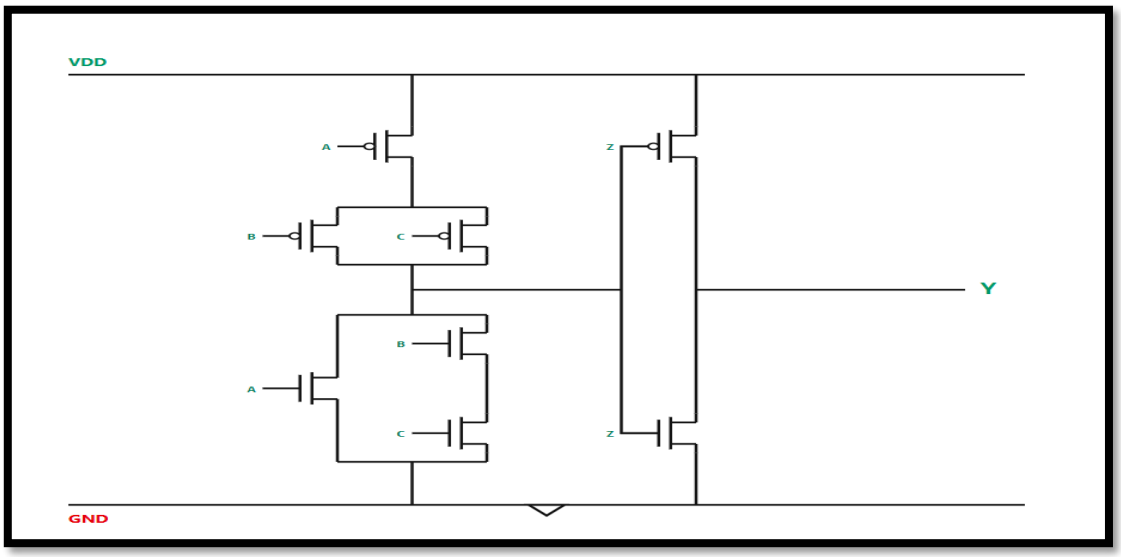
$= A + AC + AB + BC + AB$

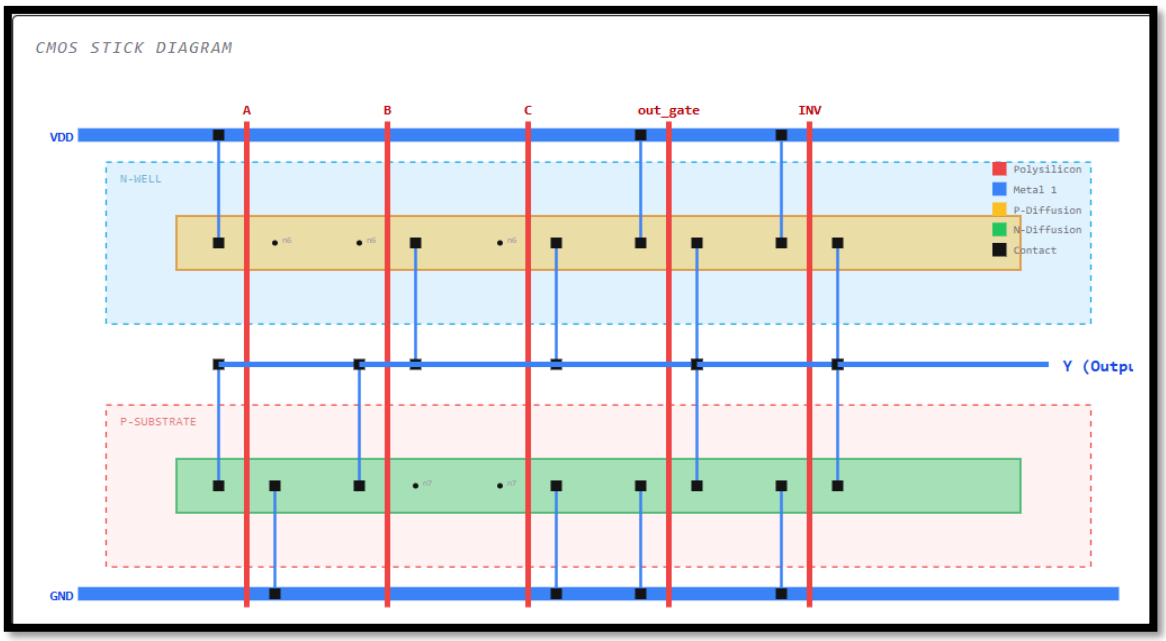
$= A(1+C) + AB + BC$

$= A + AB + BC$

$= A(1+B) + BC$

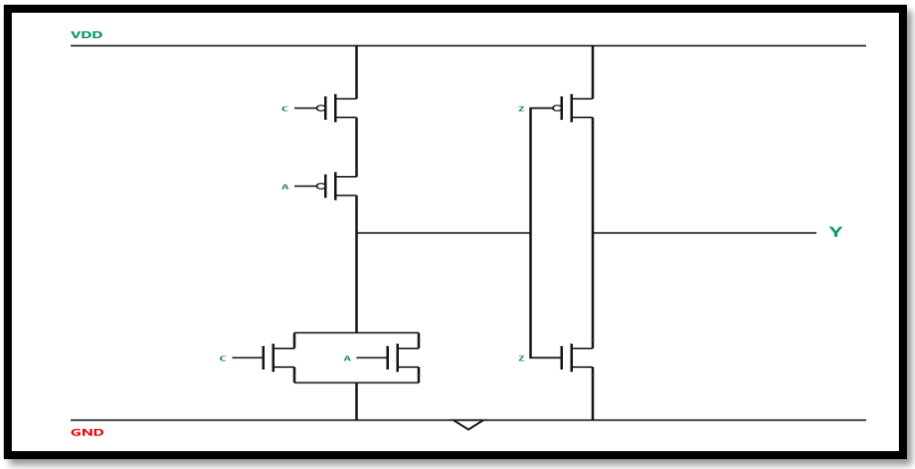
$= A + BC$

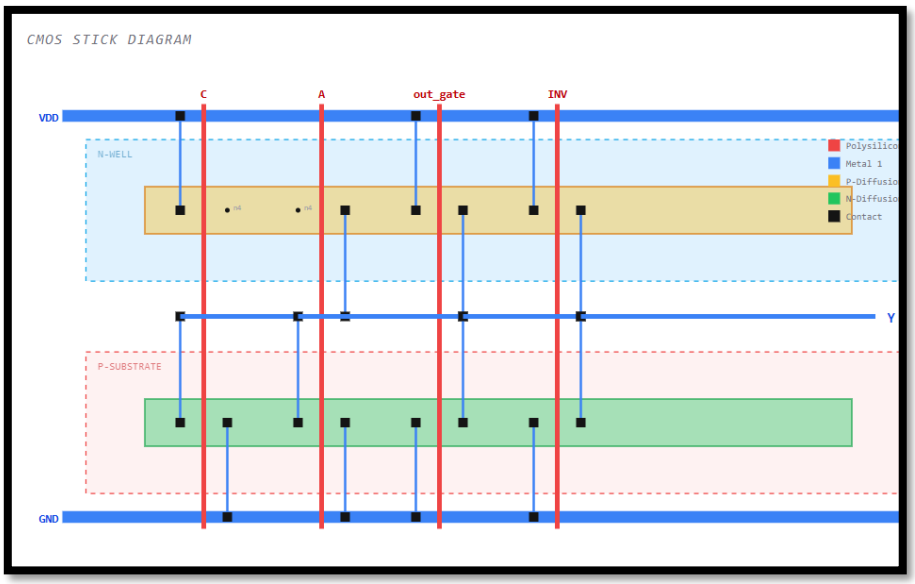




9. **F = A.B + (A+C). (A.B)' – [DE Morgan's Law, Complementary Law, Distributive Law, Idempotent Law]**

$$F = AB + (A+C).\overline{AB}$$
$$= AB + (A+C).(\bar{A} + \bar{B})$$
$$= AB + A\bar{A} + A\bar{B} + \bar{A}C + \bar{B}C + A\bar{A}$$
$$= AB + A\bar{B} + \bar{A}C + \bar{B}C + A\bar{A} \to 0$$
$$= AB + \bar{A}C + A\bar{B} + \bar{B}C$$
$$= A(B + \bar{B}) + \bar{A}C + \bar{B}C$$
$$= A + \bar{A}C + \bar{B}C$$
$$= A + C + \bar{B}C$$
$$= A + C(1 + \bar{B})$$
$$= A + C$$





10. F = (A+A'). (B+B') – [Complementary Law, Distributive Law]

$$F = (A+\bar{A})(B+\bar{B})$$

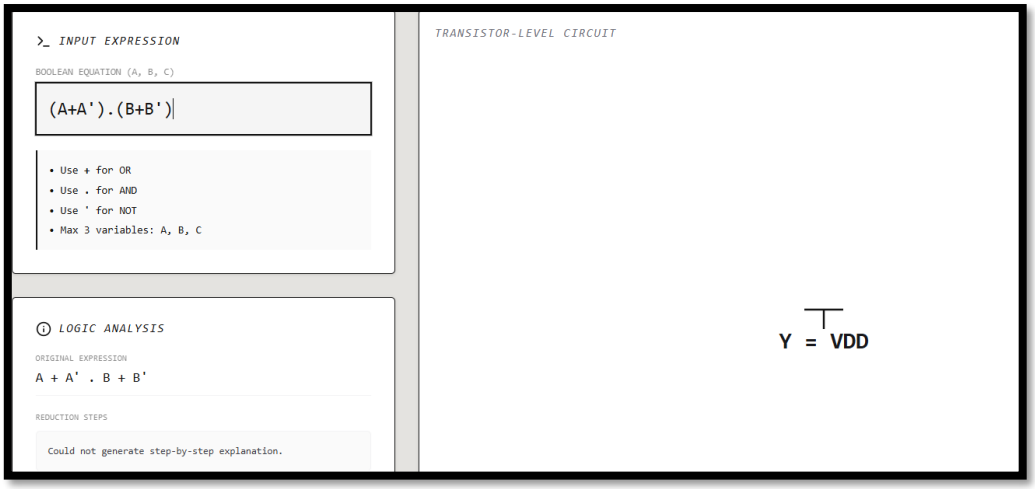
$$= AB + A\bar{B} + \bar{A}B + \bar{A}\bar{B}$$

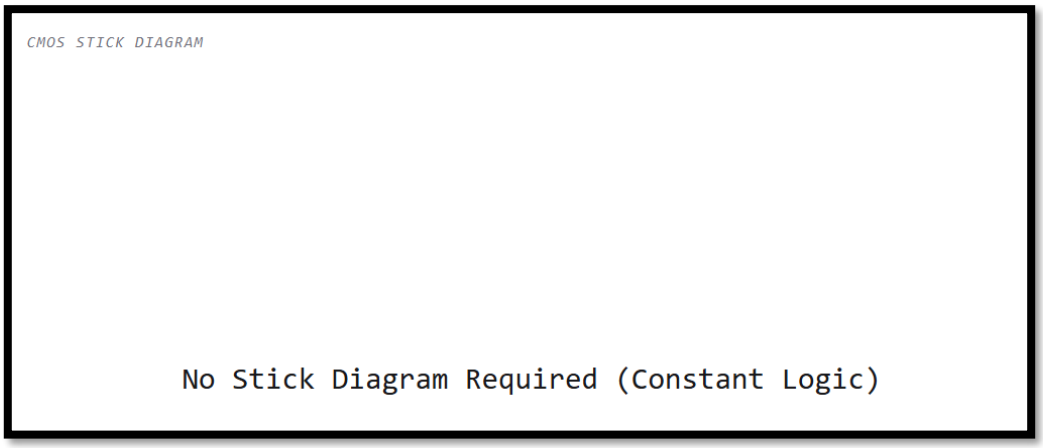
$$= A(B+\bar{B}) + \bar{A}(B+\bar{B})$$

$$= A + \bar{A}$$

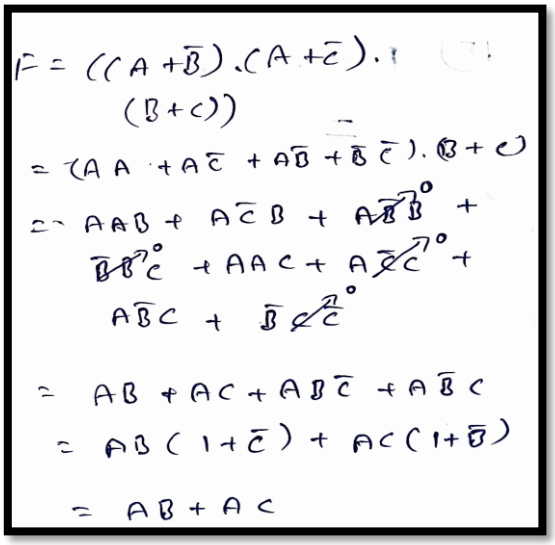
$$F = 1$$

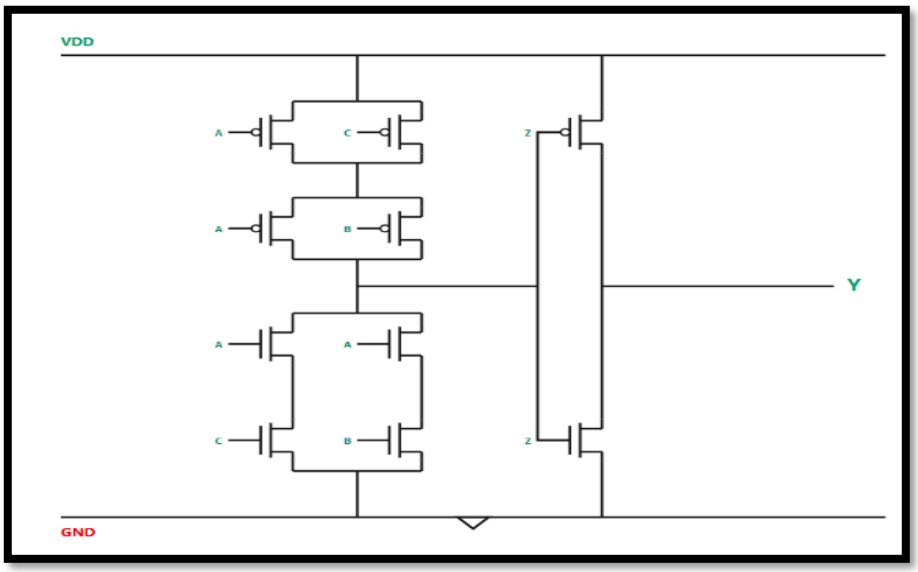
Here the output is showing VDD as the simplified expression of the expression is '1'. So, there is no circuit nor the stick diagram.

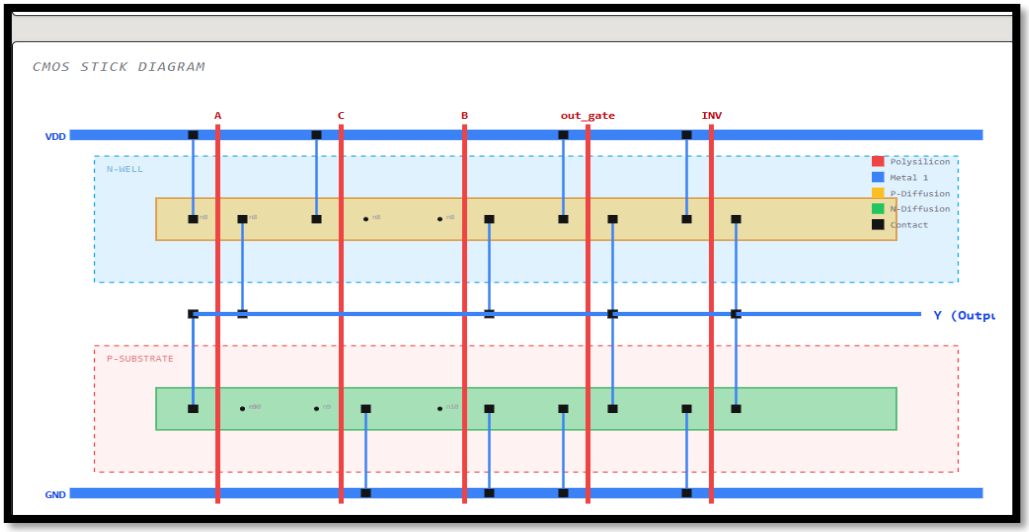




11. **F = ((A+B'). (A+C'). (B+C)) -- [Complementary Law, Distributive Law, Law of Union, Idempotent Law]**

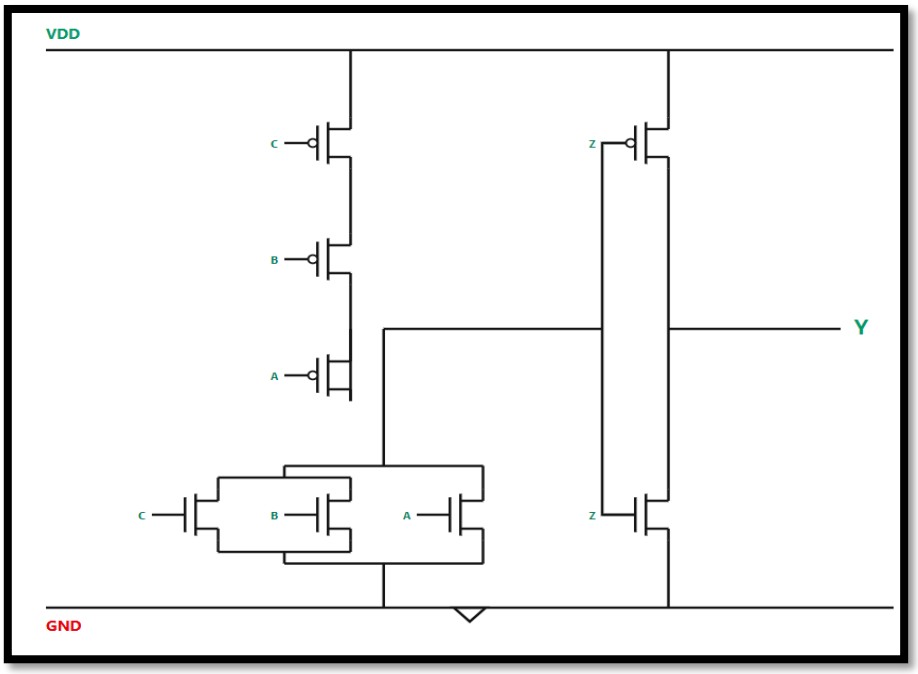


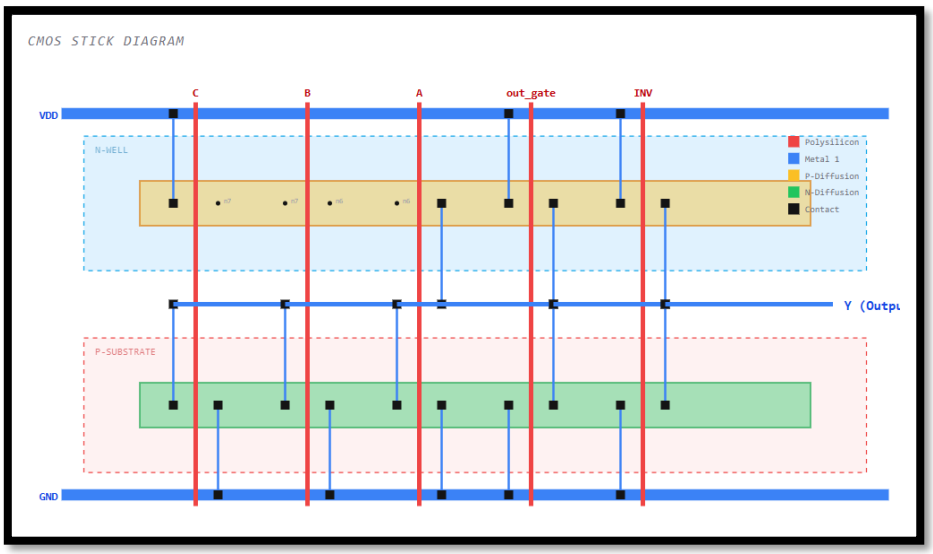




12. F = (A.B +C) +(A+B+C) – [Distributive Law, Law of Union, Idempotent Law]

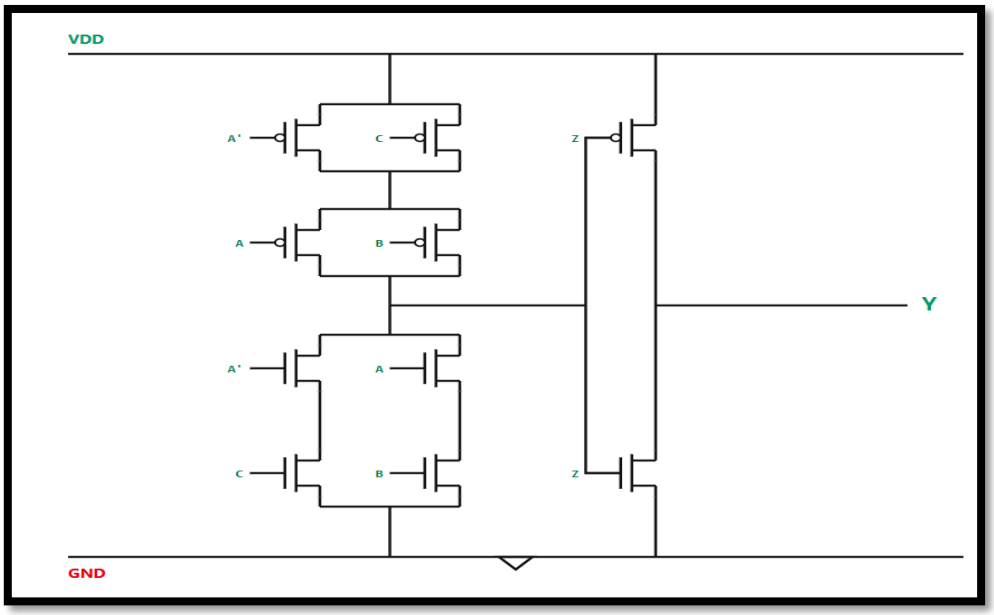
$$F = (AB+C)+(A+B+C)$$
$$= ABA + ABB + ABC + AC + BC + CC$$
$$= AB + AB + AC + BC + ABC + C$$
$$= AB(1+C) + AC + BC + C$$
$$= AB + C(1+A) + BC$$
$$= AB + C(1+B)$$
$$= AB + C \Rightarrow A \cdot B + C$$

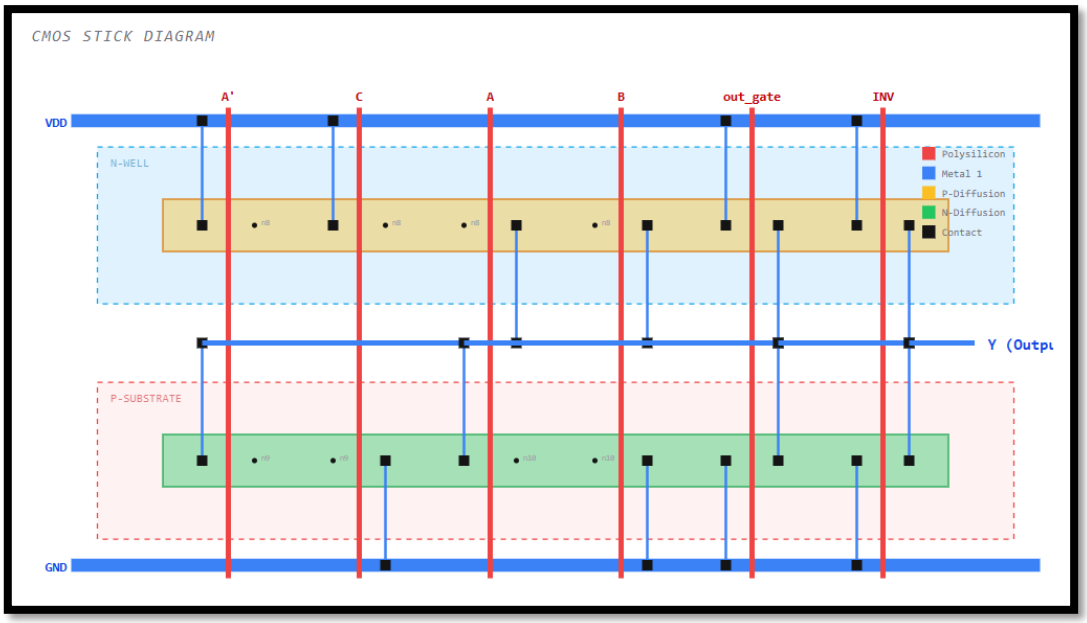




13. F= ((A.B) +(A'.C) + (B.C)) – [Law of Union, Idempotent Law]

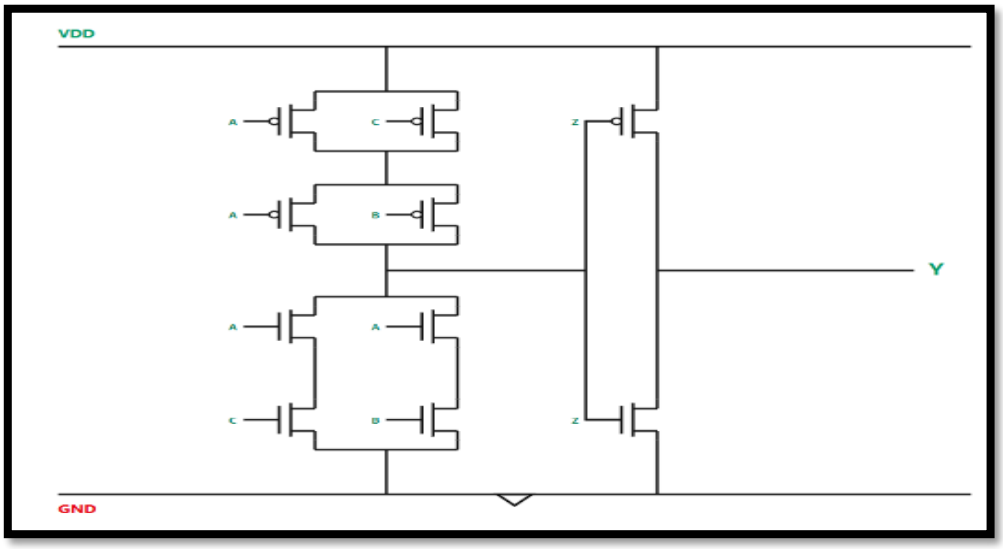
$$F = ((AB) + (\bar{A}C) + (BC))$$
$$= AB + \bar{A}C + BC$$
$$= B(A + C) + \bar{A}C$$
$$= \bar{A}CC + AB$$
$$= AB + \bar{A}C$$

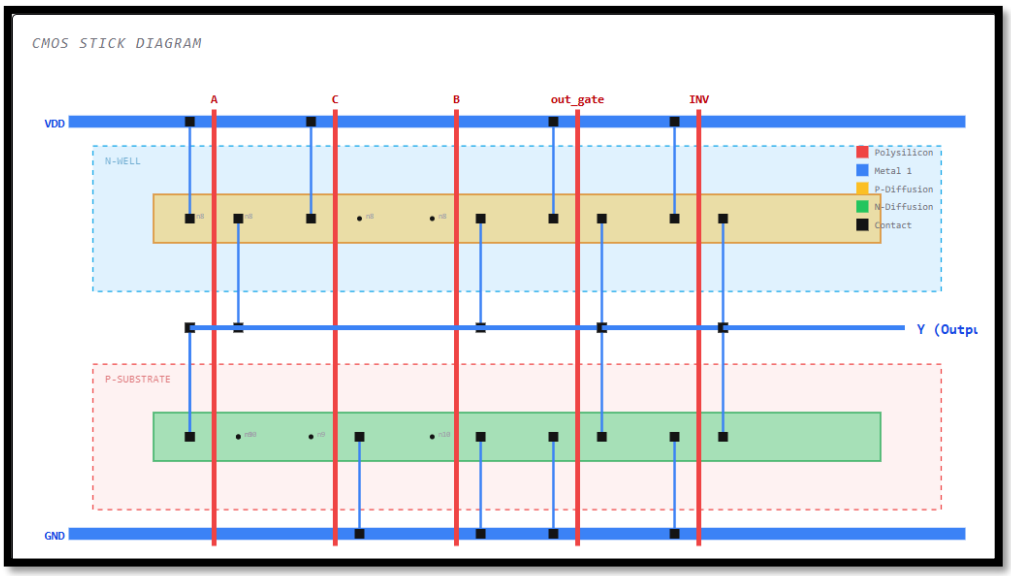




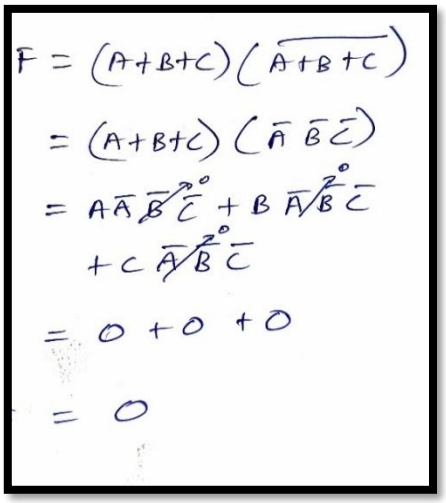
14. **F= ((A+B'). (A+C'). (B+C) – [Complementary Law, Distributive Law, Law of Union, Idempotent Law]**

$$F = ((A + \bar{B})(A + \bar{C})(B + C))$$
$$= (AA + A\bar{C} + A\bar{B} + \bar{B}\bar{C})(B + C)$$
$$= AAB + A\bar{C}B + A\bar{B}B + \bar{B}\bar{C}B + AAC + A\bar{C}C + A\bar{B}C + \bar{B}\bar{C}C$$
$$= AB + AC + AB\bar{C} + A\bar{B}C$$
$$= AB(1 + \bar{C}) + AC + A\bar{B}C$$
$$= AB + AC$$

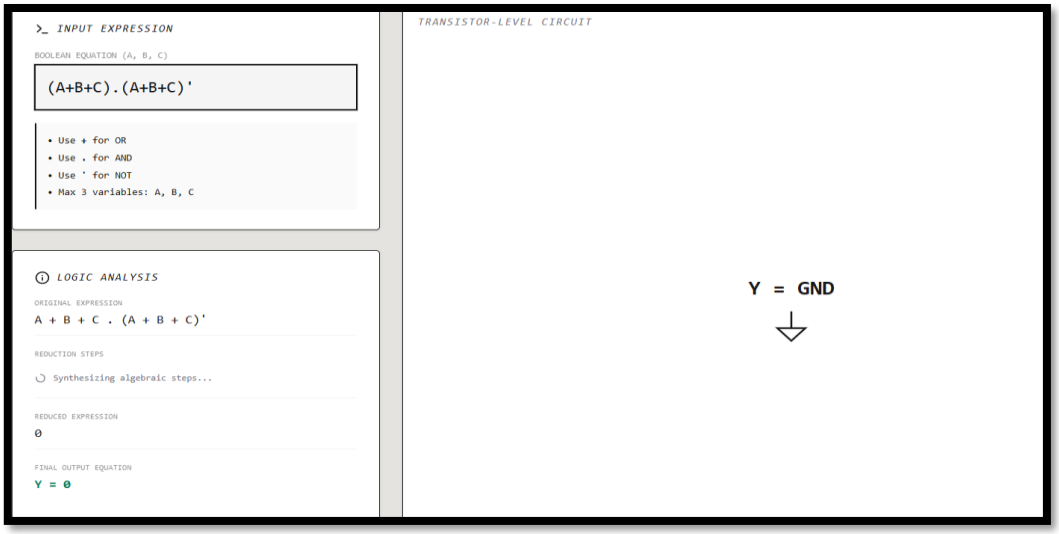


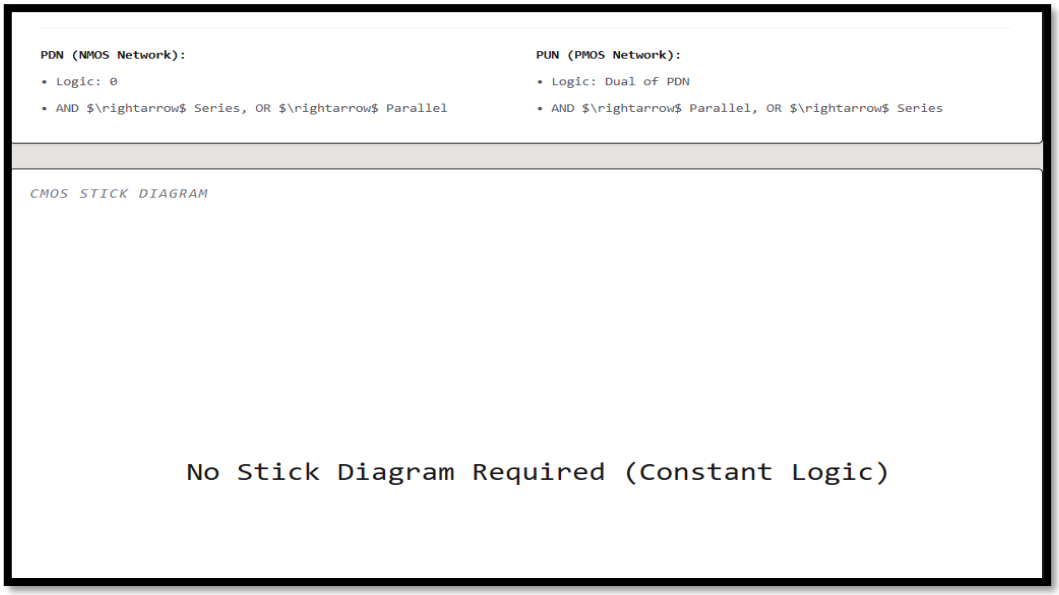


15. F =(A+B+C). (A+B+C)'– [Complementary Law, Distributive Law]



Here the output is showing GND as the simplified expression of the expression is '0'. So, there is no circuit nor the stick diagram.





A.1 Glossary of Key Parameters

Table A.1 defines the important parameters and terminology used in CMOS circuit generation and transistor-level implementation. Table A.1 shows the important parameters and terminology pertinent to the generation of CMOS circuits and the implementation in transistors. These terms are foundation of the CMOS Logic Circuit Visualizer system and assist in understanding the transformation of Booleans to digital circuits. Boolean is a mathematical representation of logic operations implemented in digital logic (AND, OR, NOT etc) with variables. These expressions are run through to create CMOS transistor circuits. Pull-Up Network (PUN) is made up using PMOS transistors and in CMOS design it is used for connecting the output to power supply to create logic HIGH output. Likewise, the Pull-Down Network (PDN) is used to pull the output signal to ground and to provide a logic LOW output signal. Both transistors can conduct when the gate input has a certain threshold, such as a pull up function for PMOS or a pull-down function for NMOS. These transistors are grouped in the manner to provide the logic function of the circuit. A transistor series circuit is an AND gate as they all have to turn on at once to form a path. By contrast, a parallel connection is an 'OR' operation since there is always a path for a conducting transistor to complete the circuit. Another use of complement logic are CMOS circuits in which the PMOS network is the logical complementary network of the NMOS network, serving to ensure proper and efficient performance.

Table A.1: Glossary of CMOS Design Parameters

Parameter	Description
Boolean Expression	Mathematical representation of digital logic operations
Pull-Up Network (PUN)	PMOS transistor network responsible for logic HIGH output
Pull-Down Network (PDN)	NMOS transistor network responsible for logic LOW output
PMOS Transistor	Transistor used in the pull-up network that conducts for LOW gate input
NMOS Transistor	Transistor used in the pull-down network that conducts for HIGH gate input
Series Connection	Transistor arrangement representing AND logic operation
Parallel Connection	Transistor arrangement representing OR logic operation

Complement Logic	Inverse Boolean logic used for PMOS network generation
Input Variables	User-defined logic inputs such as A, B, C, and D
Output Node	Final logic output generated by the CMOS circuit
Logic Gate	Fundamental digital building block such as AND, OR, and NOT
CMOS Technology	Complementary Metal Oxide Semiconductor design methodology
Transistor-Level Design	Circuit representation using individual MOS transistors
Circuit Visualization	Graphical representation of the generated CMOS structure
GUI Interface	User interface used to input Boolean expressions

A.2 Selected Code Snippet: CMOS Circuit Generation

The following Python function demonstrates the logic used to generate CMOS transistor networks from a Boolean expression entered by the user.

```python
def generate_cmos(expression):
    pull_down = create_nmos_network(expression)
    complement_expr = complement(expression)
    pull_up = create_pmos_network(complement_expr)

    circuit = {
        "PMOS": pull_up,
        "NMOS": pull_down
    }

    return circuit
```

Figure A.2: Generated CMOS circuit output

Listing A.1: Python Code for CMOS Network Generation

The function performs the following operations:

1. Generates the NMOS pull-down network directly from the Boolean expression.
2. Generates the complementary PMOS pull-up network.
3. Combines both transistor networks into a complete CMOS circuit structure.
4. Returns the transistor-level representation for visualization.

A.3 Supplementary Figures

Figure A.1: CMOS Logic Generator Input Interface

The graphical user interface allows users to:

1. Enter Boolean expressions
2. Select logic operations
3. Generate transistor-level CMOS circuits
4. Visualize PMOS and NMOS arrangements

Figure A.2: Generated CMOS Circuit Output

The generated output diagram displays:

1. Pull-Up PMOS Network
2. Pull-Down NMOS Network
3. Series transistor arrangements for AND logic
4. Parallel transistor arrangements for OR logic
5. Input and output node connections

A.4 Dataset and Input Features

The following logical elements and transistor parameters are considered during CMOS circuit generation:

1. Boolean Variables
2. AND Operations

3. OR Operations
4. NOT Operations
5. PMOS Transistor Mapping
6. NMOS Transistor Mapping
7. Series Network Construction
8. Parallel Network Construction
9. Complement Logic Generation
10. Output Node Assignment
11. Gate-Level Translation
12. Transistor Interconnection
13. Logic Simplification
14. Input Parsing
15. Circuit Visualization Coordinates
16. Graph Rendering Parameters
17. Transistor Labels
18. Connection Routing
19. Pull-Up Voltage Source (VDD)
20. Ground Connection (GND)

A.5 Software and Development Tools

The CMOS Logic Circuit Generator was implemented using the following technologies:

Table A.5: Tools using CMOS Design Software

Software / Tool	**Purpose**
Python	Core programming language
Tkinter	GUI development
Graphviz	CMOS circuit visualization
PIL / Pillow	Image rendering and processing
VS Code	Source code development
GitHub	Version control and project management

A.6 Applications of CMOS Circuit Generator

The developed system can be applied in the following areas:

1. Digital Electronics Education
2. VLSI Design Learning
3. CMOS Logic Verification
4. Transistor-Level Circuit Analysis
5. Logic Gate Visualization
6. Semiconductor Design Automation
7. Academic Research and Demonstration

A.7 Future Enhancements

Future improvements to the CMOS Logic Circuit Generator may include:

1. Automatic Boolean expression simplification
2. Support for XOR and XNOR gate generation
3. Integration with Verilog HDL
4. Real-time simulation of logic outputs
5. Power and delay analysis
6. Web-based deployment for online access
7. Support for large-scale combinational circuits
8. Export functionality for PDF and image formats

This appendix includes additional technical information and references for the CMOS Logic Circuit Generator project. The code snippets, the description of the parameters and the structured explanations help with the reproducibility and to better understand the transistor-level CMOS logic design.

References

[1] C. E. Shannon, “A symbolic analysis of relay and switching circuits,” *Transactions of the American Institute of Electrical Engineers*, vol. 57, no. 12, pp. 713–723, 1938, Doi: 10.1109/T-AIEE.1938.5057767.

[2] M. Karnaugh, “The map method for synthesis of combinational logic circuits,” *Transactions of the American Institute of Electrical Engineers*, vol. 72, no. 5, pp. 593–599, 1953, Doi: 10.1109/TCE.1953.6371932.

[3] J. M. Rabaey, A. Chandrakasan, and B. Nikolic, “Digital integrated circuits: A design perspective,” *Prentice Hall Electronics and VLSI Series*, 2003, doi: 10.5555/861369.

[4] N. H. E. Weste and D. Harris, “CMOS VLSI design: A circuits and systems perspective,” *Addison-Wesley Publishing Company*, 2011, doi: 10.5555/2350252.

[5] Y. Taur and T. H. Ning, “Fundamentals of modern VLSI devices,” *Cambridge University Press*, 2009, Doi: 10.1017/CBO9780511811216.

[6] S. M. Kang and Y. Leblebici, “CMOS digital integrated circuits: Analysis and design,” *McGraw-Hill Education*, 2003, Doi: 10.1036/0072460539.

[7] P. W. Tuinenga, “SPICE: A guide to circuit simulation and analysis using PSpice,” *Prentice Hall*, 1995, Doi: 10.5555/230451.

[8] G. De Micheli, “Synthesis and optimization of digital circuits,” *McGraw-Hill Education*, 1994, Doi: 10.5555/175287.

[9] M. Bostock, V. Ogievetsky, and J. Heer, “D3: Data-driven documents,” *IEEE Transactions on Visualization and Computer Graphics*, vol. 17, no. 12, pp. 2301–2309, 2011, doi: 10.1109/TVCG.2011.185.

[10] M. Grinberg, “Flask web development: Developing web applications with Python,” *O’Reilly Media*, 2018, Doi: 10.5555/3185538.

[11] D. Harris and S. Harris, “Digital design and computer architecture,” *Morgan Kaufmann Publishers*, 2016, Doi: 10.1016/C2014-0-04544-3.

[12] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, “Compilers: Principles, techniques, and tools,” *Pearson Education*, 2006, Doi: 10.5555/1177220.

[13] E. R. Tufte, “The visual display of quantitative information,” *Graphics Press*, 2001, Doi: 10.2307/1576212.

[14] R. Jain and M. Balakrishnan, “Interactive learning platforms for VLSI education,” *IEEE Transactions on Education*, vol. 48, no. 4, pp. 735–742, 2005, Doi: 10.1109/TE.2005.852602.

[15] S. Hassoun and T. Sasao, “Logic synthesis and verification,” *Springer Science & Business Media*, 2002, Doi: 10.1007/978-1-4615-1049-0.

[16] J. Bhasker, “A VHDL primer,” *Prentice Hall PTR*, 1999, Doi: 10.5555/553916.

[17] B. Shneiderman and C. Plaisant, “Designing the user interface: Strategies for effective human-computer interaction,” *Pearson Education*, 2010, Doi: 10.5555/1863095.

[18] R. Buyya, C. S. Yeo, and S. Venugopal, “Market-oriented cloud computing: Vision, hype, and reality for delivering IT services as computing utilities,” *Future Generation Computer Systems*, vol. 25, no. 6, pp. 599–616, 2009, Doi: 10.1016/j.future.2008.12.001.

[19] T. Anderson and F. Elloumi, “Theory and practice of online learning,” *Athabasca University Press*, 2004, Doi: 10.15215/aupress/9781897425080.01.

[20] K. Keutzer, A. Newton, J. Rabaey, and A. Sangiovanni-Vincentelli, “System-level design: Orthogonalization of concerns and platform-based design,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 19, no. 12, pp. 1523–1543, 2000, Doi: 10.1109/43.891450.

[21] S. M. Sze and K. K. Ng, “Physics of semiconductor devices,” *Wiley-Interscience*, 2006, Doi: 10.1002/0470068329.

[22] D. Harel and Y. Feldman, “Algorithmics: The spirit of computing,” *Addison-Wesley*, 2004, Doi: 10.5555/1207724.

[23] I. Sommerville, “Software engineering,” *Pearson Education*, 2015, Doi: 10.5555/2737595.

[24] A. Dix, J. Finlay, G. D. Abowd, and R. Beale, “Human-computer interaction,” *Pearson Education*, 2004, Doi: 10.5555/1208288.

[25] R. H. Katz and G. Borriello, “Contemporary logic design,” *Pearson Education*, 2004, Doi: 10.5555/1200621.